GE Digital

Habitat

5.12

Habitat User Guide

Copyright

Copyright 2023 General Electric Company and/or its affiliates (“GE”). All Rights Reserved. This document is the confidential and proprietary information of GE and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE.

Change History

Date	Change Description
May 22, 2023	Habitat 5.12: • Editorial and formatting updates.

Contents

About This Document ..... **viii**
Purpose of This Document viii
Who Should Use This Document viii
For More Information viii
Conventions viii

1. The Habitat Product **1**
1.1 Product Overview 1
1.1.1 Habitat at a Glance 2
1.1.2 User Interface 3
1.1.3 Database Management 4
1.1.4 Run-Time Environment 4
1.1.5 User Interface Development Environment 5
1.1.6 Programming Environment 6
1.2 Habitat Architecture 7
1.3 Habitat Group 7

2. Habitat and the Operating System **8**
2.1 Habitat Accounts and Groups 8
2.2 Habitat Directory Structure 8
2.3 Habitat Environment Variables 11
2.4 File Protection and Ownership 12
2.5 File Naming Conventions 13
2.6 Remote Users 13
2.7 Habitat in Windows 13

3. Controlling and Configuring Habitat **14**
3.1 Overview 14
3.1.1 Login Sequence 14
3.1.2 Startup Sequence 14
3.1.3 Shutdown Sequence 14
3.2 Login/Startup/Shutdown Sequence 15
3.2.1 Login Sequence on Linux 15
3.2.2 Startup Sequence on Linux 15
3.2.3 Shutdown Sequence on Linux 16
3.2.4 Login Sequence on Windows 16
3.2.5 Startup Sequence on Windows 16
3.2.5.1 Startup Sequence on Windows When Not Running as a Service 16
3.2.5.2 Startup Sequence on Windows When Running as a Service 16
3.2.6 Shutdown Sequence on Windows 17
3.3 Customizing Your Habitat Environment 17
3.3.1 Customizing Startup Tasks 17
3.3.2 Customizing the Habitat Environment Variables 17
3.3.2.1 Adding User Environment Variables 18
3.3.2.2 Modifying Habitat Environment Variables 18
3.3.3 Customizing the Login Procedure 19
3.4 Linux-Specific Issues 19
3.5 Windows-Specific Issues 20
3.6 Preserving Your Changes through Upgrades 20

4. Habitat Operations 21
4.1 Habitat Startup and Shutdown 21
4.1.1 Starting Habitat 21
4.1.2 Stopping Habitat 21
4.2 Setting the Application and Family Context 22
4.3 Navigating Through Habitat Directories 22
4.4 Monitoring and Controlling Habitat Tasks 23
4.4.1 Controlling Tasks with TVIEW 23
4.4.1.1 The List Command 23
4.4.1.2 The Run Command 24
4.4.1.3 The Stop Command 24
4.4.1.4 Running Tasks Interactively 25
4.4.2 Viewing Message Logs with LISTMSG 25
4.4.3 Running Tasks and Paths from the Command Line 26
4.5 Managing Clones and Savecases 26
4.5.1 The HdbCloner Utility 26
4.5.1.1 Displaying Clone Information 27
4.5.1.2 Creating a New Clone 27
4.5.1.3 Removing an Existing Clone 28
4.5.1.4 Updating a Clone 28
4.5.2 The HdbCopydata Utility 28
4.5.2.1 Savecases vs. Archives 29
4.5.2.2 Creating Savecases and Archives 29
4.5.2.3 Retrieving Savecases and Archives 30
4.5.2.4 Displaying Available Savecase Types 30
4.5.2.5 Managing Savecases from the UI 31
4.5.3 The HdbRio Utility 32
4.5.3.1 Examine the Records in a Database 33
4.5.3.2 Navigating in HDBRIO 33
4.5.3.3 Reading and Modifying Fields 34
4.5.3.4 Inserting and Deleting Records 35
4.5.3.5 Database Maintenance 36

5. Habitat Management 37
5.1 Managing a System with the "habitat" Command 37
5.1.1 The Habitat Command Interface 37
5.1.2 Habitat Command Overview 38
5.1.2.1 The Start Command 38
5.1.2.2 The Stop Command 38
5.1.2.3 The Update Command 39
5.1.2.4 The Help Command 39
5.1.2.5 The Build Command 39
5.1.2.6 The Link Command (Linux Only) 40
5.1.2.7 The Load Command 41
5.1.2.8 The Clone Command 42
5.2 Performing a System Build 43
5.2.1 Integrating a New or Changed Alarm Message or Status File 43
5.2.1.1 Alarm Message (.almmsg) Files 43
5.2.1.1.1 Linux Almmsg Processing 43
5.2.1.1.2 Windows Almmsg Processing 43
5.2.1.2 Status (.status) Files 44

5.2.1.2.1 Linux Status Processing 44
5.2.1.2.2 Windows Status Processing 44
5.3 Managing Clones in Habitat 44
5.3.1 Creating Additional Clone Directories 44
5.3.2 Increasing the Maximum Number of Clones 45
5.4 Customizing Habitat Applications 45
5.4.1 Customizing Database Sizes 46
5.4.2 Modifying Application Quotas and Privileges 46
5.4.3 Localizing Messages and Status Values 47

6. User Interface Management 48

7. PROCMAN and the "HABITAT" PROCMAN Model 49
7.1 The Role of PROCMAN 49
7.2 The "HABITAT" PROCMAN Model 49
7.3 Customizing the PROCMAN Model 50
7.4 Habitat Tasks and Their Relationships with CFGMAN and PROCMAN 50

8. Redundant Instances of Habitat 53
8.1 Dual-Redundant Systems 53
8.2 Data Recovery and Replication 54
8.2.1 Startup and Recovery 54
8.2.2 Replication and Failover 55
8.3 Managing Redundant Systems in Habitat 56
8.3.1 Configuration Manager 56
8.3.2 MRS 57
8.3.3 PROCMAN 57
8.3.4 Alarm 57

9. Other Habitat Applications 59
9.1 Configuration Manager 59
9.2 Alarm 59
9.3 NETIO 59
9.4 Logman 60
9.5 Sampler 60
9.6 PLCMGR 60
9.7 Casesrv 61
9.8 Timedate 61
9.9 Trend 61

10. Software Configuration Management 62
10.1 Organization of Code in a Habitat Environment 62
10.2 Customizing a Habitat Release 62
10.2.1 Customizing Savecases 63
10.2.2 Customizing Database Definition (DBDEF) Files 63
10.2.3 Customizing Application Definition (APPDEF) Files 63
10.2.4 Customizing User Interface (DDL) Files 64
10.2.5 Adding New Applications to the System 64
10.2.6 Customizing Alarm 65

11. Frequently Asked Questions 66
11.1 How To 66

11.2 Tools ... 66
11.2.1 hdbcloner ... 66
11.2.2 HdbCopydata ... 67
11.2.3 HdbRio ... 67
11.2.4 HdbDump ... 68
11.2.5 Habitat Test Programs ... 68
11.3 Common Problems and Questions ... 68
11.3.1 Habitat Fails to Start ... 68
11.3.2 How to Prevent Habitat from Rebooting the Local CPU ... 68
11.3.3 DEBUG Environment Variables ... 69
11.3.4 How to Move Clones ... 69
11.3.5 Where can I find the version of Habitat? ... 69
11.3.6 Can I modify the database size of a Habitat-owned application? ... 69
11.3.7 Why is my recovery set not getting recovered? ... 69

12. Linux-Specific Issues ... 71
12.1 Changing the Maximum Number of Clones ... 71
12.2 Understanding the Habitat UPDATE Process ... 71

13. Windows-Specific Issues ... 73
13.1 The Habitat Product ... 73
13.2 Habitat as a Service ... 73
13.2.1 Overview ... 73
13.2.2 Advantages ... 74
13.2.3 Habitat Accounts and Groups ... 75
13.2.4 Starting and Stopping ... 75
13.2.5 TVIEW ... 76
13.3 Habitat Environment Variables ... 77
13.3.1 Overview ... 77
13.3.2 Environment Variables in Windows ... 77
13.3.2.1 Customizing Environment Variables ... 78
13.3.2.2 Multiple Instances of Habitat ... 78
13.4 TVIEW ... 78
13.4.1 Using Shortcuts to Automate Startup ... 81
13.4.2 Special Privilege to run TVIEW on Windows ... 81
13.5 Configuring Habitat to Start after Rebooting ... 81
13.6 Non-Standard Network Considerations (Laptops) ... 82

Figures

Figure 1. Habitat at a Glance ... 3
Figure 2. Habitat Architecture ... 7
Figure 3. HABITATgg Subtree ... 9
Figure 4. HABSRCgg Subtree ... 9
Figure 5. HABMGRgg Subtree ... 10
Figure 6. HABDATAgg Subtree ... 10
Figure 7. HABWEBgg Subtree ... 11
Figure 8. Savecase Master Display ... 31
Figure 9. Savecase Application Display ... 32
Figure 10. Dual-Redundant Configuration ... 53

Figure 11. Startup 55
Figure 12. Data Replication 56
Figure 13. Alarm Interaction 58
Figure 14. TVIEW Interacting with Habitat Running as a Service 76
Figure 15. TVIEW Interacting with Habitat Not Running as a Service 77
Figure 16. TVIEW Graphical User Interface 79
Figure 17. TVIEW Main Menu 80
Figure 18. TVIEW Right-Click Menu 81

About This Document

This document is supplied as a part of GE's Habitat product.

Purpose of This Document

This document provides instructions for performing typical Habitat operations. Several of the key features of Habitat are explained, including redundancy, software configuration management, and system configuration.

Who Should Use This Document

This document is intended for all Habitat users, including system managers, integrators, development engineers, and casual users.

For More Information

For more information about Habitat and its components, refer to the following:

- *Habitat Release Notes*: Specific installation information and requirements for the current Habitat release.
- *Habitat Software Installation and Maintenance Guide*: Provides information about installing and configuring Habitat.
- *Browser Software Installation and Maintenance Guide*: Provides information about installing and configuring Browser.

For more information about Microsoft Windows, refer to the following:

- Microsoft Press Store Web site: www.microsoftpressstore.com/store/browse/windows.

For more information about the Red Hat Linux operating system, refer to the following:

- www.redhat.com/software/rhel
- www.redhat.com/docs/

Conventions

The following conventions are used throughout this document.

Unless contained in an operating system–specific example, directory specifications are shown using Linux file system conventions. For example:

`/eterra/habitat60/habitat/bin`

is equivalent to the Windows directory specification:

`\Eterra\habitat60\habitat\bin`

1. The Habitat Product

This chapter provides an overview of the Habitat product.

1.1 Product Overview

Habitat is a software environment that supports the development and operation of highly available, real-time control systems. The types of real-time control systems for which Habitat is primarily targeted have the following basic attributes in common:

- Large amounts of fast-changing, real-time data: Habitat's database subsystem is designed to support large databases, where tens of thousands of measurements are scanned periodically (a 2- to 10-second rate) and the user interface must refresh the data on the operators' screens at rates ranging from 2 to 10 seconds.
- A requirement for a responsive user interface: With such large amounts of rapidly changing data, it is critical that the operators of the system be able to monitor and view the data. Habitat's user interface subsystem provides a highly responsive tool-based user interface that can support large geographic and one-line displays as well as database-driven tabular displays.
- High availability: These systems run 24 hours a day, 365 days a year; there is no scheduled down time. At a basic level, this usually means that there cannot be any single point of failure of software or hardware. Thus, systems are totally redundant: servers, networks, disks, etc. Habitat includes software to monitor the health and well-being of the hardware and software in the system, and it uses knowledge of which components are redundant and their current states in order to maximize the availability of the overall control system.

Beyond these functional requirements, these control systems are usually critical business assets and, as such, they must evolve and be maintained in order to keep up with changing technology. Three key attributes of Habitat ensure that these systems stay current:

- A product approach: Habitat is developed and delivered as a product. Documented design, implementation, and test processes are followed to ensure that customers get a product that is ready to work in the demanding systems described above. These same processes and the associated management of the product ensure that Habitat is stable and reliable. The fact that it is a product also means that all customers gain from feedback such as suggestions, experiences, and problem reports into the system.
- A migration philosophy: Since its very beginning, Habitat has been constructed with migration (i.e., periodic upgrades) in mind. At periodic intervals, new versions of Habitat are released with the fundamental attribute that they must support future upgrades. Customers never lose their investment in their databases, displays, or custom software.
- Portability: Habitat is portable. It is available for real-time use in Linux and Windows. Portability means many things: It means that you can develop and test your applications on one operating system (for example, using Visual Studio in Windows) and deploy it on another. It also means that you have a built-in insurance policy protecting you against obsolescence of your operating system. Because your databases, user interface, and applications are portable, changing operating systems does not require the procurement of a new control system.

The remainder of this chapter provides a high-level overview of Habitat so that you can understand how it satisfies the needs of highly available, real-time control systems.

1.1.1 Habitat at a Glance

The objective of this section is to provide you with a big-picture overview of Habitat. Subsequent sections go into more details about each of the main subsystems that make up Habitat.

Figure 1 provides a high-level architectural overview of Habitat.

For the purposes of this overview, Habitat can be deconstructed into the following high-level subsystems:

- User Interface: Browser provides the user interface for Habitat.
- Database Management: The database management subsystem includes support for defining database schemas, formatting them so that programs can easily access the data, and finally cloning, which creates the memory-resident database for use by applications and Browser.
- Run-Time Environment: The run-time environment includes the programs, databases, and user interfaces of the applications supplied with Habitat, plus the control system applications that make up your control system.
- User Interface Development Environment: The user interface development environment is used to create online documentation and the components that define the displays presented by Browser.
- Programming Environment: The programming environment allows you to create programs that access Habitat databases and interact with other programs as well as Browser.

The following sections describe each of the high-level subsystems.

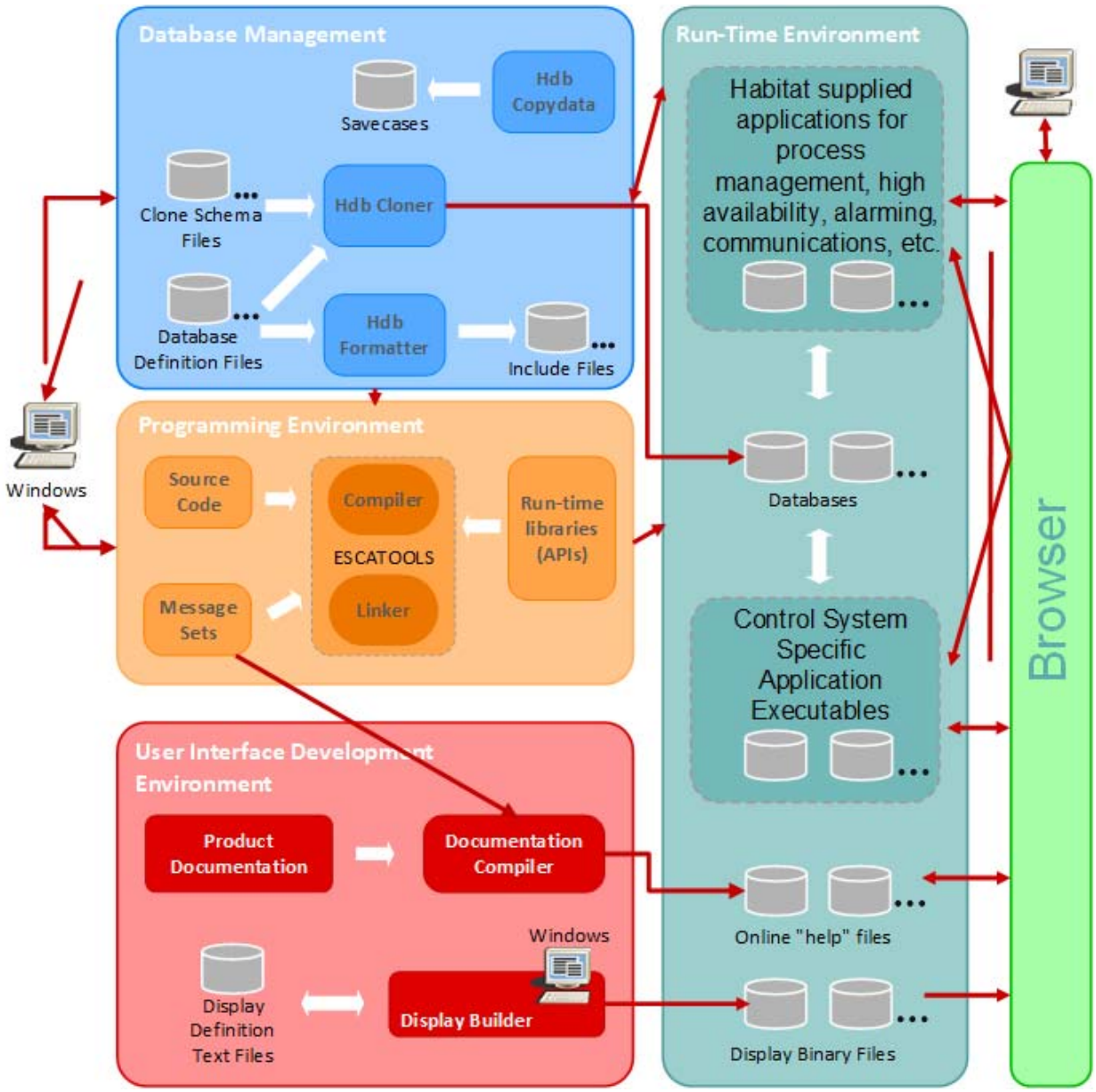

Figure 1. Habitat at a Glance

1.1.2 User Interface

Browser is Habitat's user interface subsystem. Browser uses native Windows programming methods and standard Web browser technology.

Browser uses the same Display Builder as the now-discontinued Rapport-FG. A display can be thought of as a form or a geographic map. You place picture elements on a display. A picture element is similar to a macro; it allows you to format data for a particular record type in the database. When a picture is placed on a display, you either link the picture to a specific named record in a database, or you tell Browser to repeat the picture on the display based on the number of records of that type in the respective database. The latter type of display is referred to as a "tabular display".

At run time, Browser consists of two main components: the Browser Data Server and the Browser Client. The Data Server runs on the host machine(s) where the database you wish to access resides. It is responsible for reading the data from the respective database(s) and sending it to the Client. The Client runs on a Windows workstation. It is the software with which users directly interact.

Browser achieves a much greater scalability than Rapport-FG. While Rapport-FG demanded a one-to-one correspondence between its Host-FG and Remote-FG processes, a single Browser server is capable of supplying data to many clients without loss of efficiency or performance.

1.1.3 Database Management

The database subsystem within Habitat is called “Hdb”. Habitat contains many databases rather than a single massive database. This approach is useful because it allows for excellent modularity, and it makes it very easy to develop applications without having to be concerned about impacting existing applications or their data.

There are four fundamental concepts that you must be familiar with in order to work in a Habitat environment:

- Application: An *application* is a collection of one or more databases that, together, provide storage for a particular function or collection of functions. In addition to databases, an application typically contains one or more tasks, which process the information stored in the databases. For example, within the Platform (Energy Management Platform) product, the application that is responsible for data acquisition is called “Scada”. Habitat itself is made up of several applications, such as PERMIT (the permissions subsystem), PROCMAN (Process Manager), and CFGMAN (Configuration Manager).
- Clone: A *clone* is an instance of an application. The Cloner is a tool that creates the databases for a given instance of an application. A clone schema file tells the Cloner which database schemas should be used to create a clone of the application.
- Family: When you create a clone of an application, you identify the instance of the application by a name called the clone’s *family*. The term “family” comes from a concept within Habitat that a collection of different clones that share the same family can be thought of as a set or family of applications. Within Habitat, you can have many instances of a given application.
- Context: Within Habitat, your *context* identifies what your current application clone is. A context consists of two parts: the application name and the family name.

The HdbFormat program processes database schema files (also referred to as “database definition files” or “DBDEF files”), and it generates schema-specific include files for inclusion in Fortran programs as well as header files for use in C and C++ programs.

The HdbCopydata utility provides a mechanism for creating and retrieving data from savecases. A “savecase” is a file containing data and schema information for one or more databases of a given application. Because a savecase includes schema information, it can be retrieved into an application clone even if the schema of the current clone is different from the schema at the time the savecase was made.

1.1.4 Run-Time Environment

Every Habitat system contains the same software. The only difference between a system that is used for development and one that is used for real-time data acquisition is how it is configured, i.e., which applications have been cloned and which programs are run.

The core of the run-time environment consists of the following subsystems:

- Browser: This is the user interface tool in a Habitat environment.
- Hdb: This is the subsystem that provides support for clones (databases) and the APIs that are used to access them. For more information about Hdb, see the *Hdb User Guide*.
- PERMIT: PERMIT is the subsystem that is responsible for controlling which clones and displays a given user can access and in what way the clones can be accessed (i.e., read and or read/write). For more information about PERMIT, see the *PERMIT User Guide*.
- PROCMAN: PROCMAN is the process manager within Habitat. PROCMAN is a database-driven application that is responsible for starting up, sequencing, and monitoring the tasks that run within Habitat. For more information about PROCMAN, see the *PROCMAN User Guide*.
- NETIO: NETIO is the communications subsystem within Habitat. NETIO provides services that facilitate the formation of communication links between tasks running on the local host as well as between tasks running on different hosts. For more information about NETIO, see the *NETIO User Guide*.

In addition to the core functions that are always running in any instance of Habitat, there are several other subsystems that are used primarily in real-time, highly available control systems:

- Alarm: Alarm provides an API and a sophisticated user interface to facilitate notification of operators and other staff in the event of problems in the control system or the system being controlled. For more information about Alarm, see the *Alarm User Guide*.
- Configuration Manager: Configuration Manager is responsible for monitoring the health and state of the hardware and software that make up the redundant control system. For more information about Configuration Manager, see the *Configuration Manager User Guide*.
- Hdb/MRS: In a highly available system, Hdb is capable of replicating database transactions to a secondary computer so that, in the event of a failure of the Primary, the secondary can take over with consistent data from just prior to the failure. For more information about data replication with MRS, see the *Hdb Programmer Guide* and the *Memory Replication Service User Guide*.
- PLCMGR: PLCMGR provides an API through which applications can write data to a Programmable Logic Controller (PLC). For more information about PLCMGR, see the *PLC Manager User Guide*.
- LOGMAN: LOGMAN provides support for “logging” messages to printers and queuing files. It manages and monitors the health of the printers under its control. For more information about LOGMAN, see the *Logman User Guide*.

1.1.5 User Interface Development Environment

The user interface development environment consists of three parts:

- Display Builder: A Windows-based display building tool. This application allows you to create and maintain displays for use with Browser. Although the display building files are created under the Windows operating system, they are portable to all supported operating systems. Display definitions are stored as text files in a syntax called the “Display Definition Language”; you will often see and hear about “DDL” files.
- Display Compiler Tools: The display compiler for Browser is embedded within the Display Builder.

- Documentation Tools: Although the tools themselves are not included in the standard Habitat delivery, it is possible to compile Microsoft Word–based documents into online help documents that can be accessed in conjunction with Browser.

1.1.6 Programming Environment

Habitat includes a rich set of application programming interfaces (APIs) that allow programs to interface with Habitat services such as Hdb, PROCMAN, Configuration Manager, NETIO, etc. In addition to APIs that provide access to the control system services, additional APIs are provided that enable the creation of portable applications.

If you, as a programmer, restrict yourself to the Habitat–supplied APIs and industry-standard language-specified functions, it is easy to create portable applications. By committing yourself to creating portable applications, you retain the ability to develop software on one operating system and deploy it on another. Microsoft's Visual Studio development environment is used for most development, as it has been proven to be a much more productive development environment than the standard environments provided in Linux.

The key APIs supplied with Habitat include:

- APPMGR: APPMGR stands for "application manager". APPMGR provides a portable API that encapsulates the creation of processes and the notification of their completion.
- Hdb: The HDB API provides access to Habitat databases.
- HUL: HUL stands for "Habitat Utility Library". HUL includes a collection of APIs designed to make it easier and faster to create programs. It also includes an API that, if used within an event-driven program, facilitates after-the-fact analysis (the HISTLIST API).
- Logman: The Logman API provides an interface to the Logman application. Via the Logman API, you can request that files be printed and messages be logged to a printer.
- MLF: MLF stands for "Message Logging and Formatting". MLF provides services for creating application messages, alarm formatting text, and dialog box text such that it can be changed (for example, localized) without requiring access to the application software.
- NETIO: NETIO includes three APIs that encapsulate task-to-task communications. NETIO supports TCP/IP on all operating systems.
- OSAL: OSAL stands for "Operating System Abstraction Layer". OSAL provides run-time (API) services to application programs so that they do not need to access the operating system directly. OSAL and the Habitat API standard are what make it possible to create truly portable control systems.
- PPM: PPM stands for "Portable Process Model". PPM is a portable application framework that enables the creation of portable event-driven programs.
- RUSER: The RUSER API provides communication between your application and Browser. Through this interface, your program can trap user actions initiated from your application's displays.
- Timedate: The Timedate subsystem encapsulates everything having to do with obtaining and formatting time. It also supports simulation clocks that can be associated with a given clone, so that a request for the current time within a program associated with the clone returns the simulation time.

For a more complete overview of the above APIs, refer to the *Introduction to Habitat Programming* and the *Habitat Portable API Reference Manual*.

1.2 Habitat Architecture

Habitat is a portable environment. Habitat runs in Linux and Windows, and it is possible — and the norm — to write applications that run in all environments without change. Portable applications are made possible by the portability services provided by Habitat that can be used to insulate applications from the operating system.

The following figure shows how a Habitat–based control system can be deconstructed into three layers.

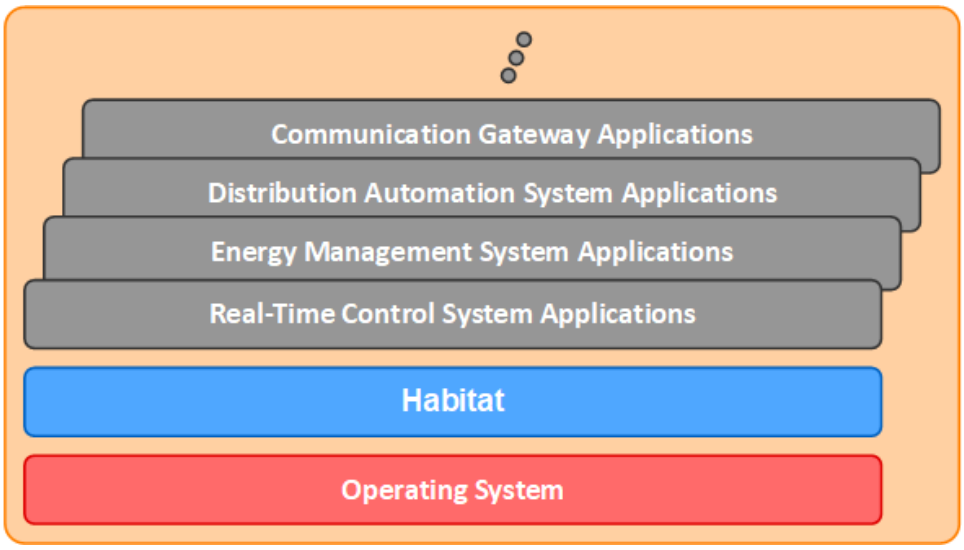

Figure 2. Habitat Architecture

Portable applications are achievable because Habitat provides all of the services that are required to support a real-time application, including many services that you would often expect to obtain directly from the operating system. Because Habitat provides these services via operating system–neutral APIs, it is possible for applications to be written without any knowledge or care as to what operating system they are being compiled on, or which operating system they are running on.

1.3 Habitat Group

Habitat is designed so that more than one instance can be installed on a machine. The directory structure and the name spaces within Habitat are associated with each other by what is called the *group*. The group name is a two-digit hexadecimal number. Typical group numbers are 15, 60, and 90, but they can be almost anything.

Habitat groups make it easy to isolate people and systems.

2. Habitat and the Operating System

This chapter discusses the relationship of Habitat to the operating system, including a discussion of the physical and logical layout of a Habitat installation.

2.1 Habitat Accounts and Groups

Every Habitat installation has an associated two-digit hexadecimal group number. In Linux, there is an associated file system group. Along with each Habitat group is an associated account; this account is named "hab*gg*", where *gg* is the group number. This is a privileged account that can perform any Habitat operation, including starting and stopping the entire instance of Habitat. When more than one Habitat group is installed on a single server, users are guaranteed that these groups stay physically and logically separate. Without action from users, the applications in the two Habitat groups never interact with one another. Specific configuration work must be done to cause an application in one Habitat group to communicate with an application in another.

In Windows, the account is not created by the Habitat installation program; it must be created manually. Although the account name is not required to be in the "habgg" format, it is often referred to as "habgg" in the product documentation. Furthermore, the installation program needs to be run by a user belonging to the Administrator group. For more information about the installation, refer to the *Habitat Software Installation and Maintenance Guide*.

2.2 Habitat Directory Structure

Habitat is laid out across several physical directories, organized along the following logical lines:

- HABITAT and HABUSER files: Those files owned by the Habitat system are physically and logically separate from those owned by layered products. The HABITAT directory tree contains the files owned by Habitat, and the HABUSER directory tree contains files owned by the layered products.
- Volatile and non-volatile files: Files that seldom change, and are relatively constant in size, are kept physically separate from data files that are volatile and changing in size.

With these organizational strategies in mind, the HABITAT directory layout becomes easily understandable. The major directory structures are illustrated in the following series of diagrams.

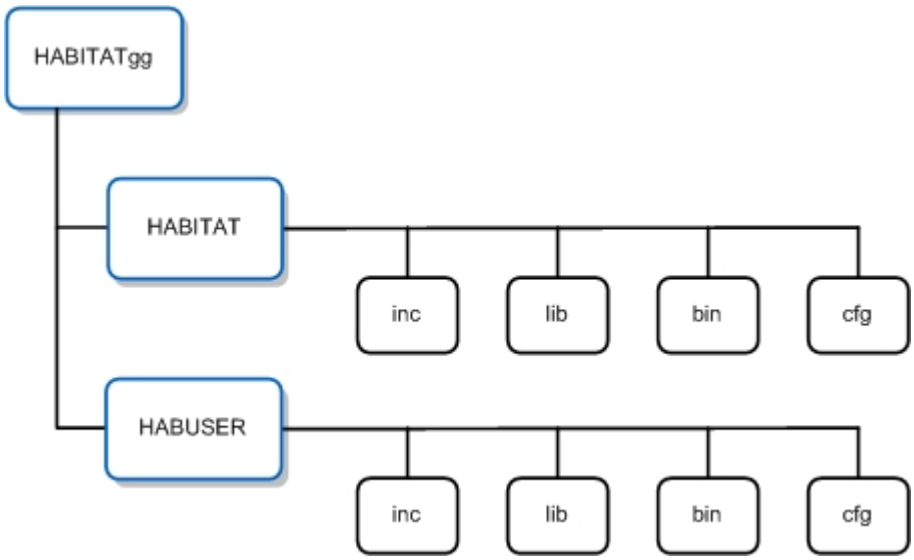

Figure 3. HABITATgg Subtree

The HABITATgg subtree contains the HABITAT and HABUSER non-volatile data.

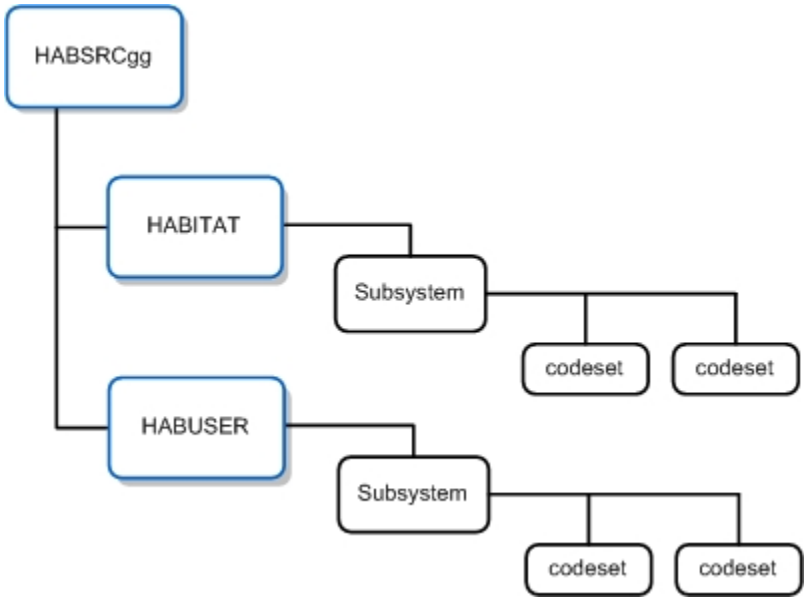

Figure 4. HABSRCgg Subtree

The HABSRCgg subtree contains the HABITAT and HABUSER codesets (source code). These codesets are organized under subsystems.

Note: There may be many subsystems, and many codesets under each subsystem.

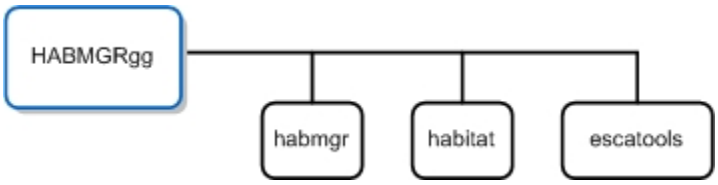

Figure 5. HABMGRgg Subtree

The HABMGRgg subtree contains the login directory for the Habitat accounts, as well as the local ESCATOOLS installation and a directory for Habitat management files.

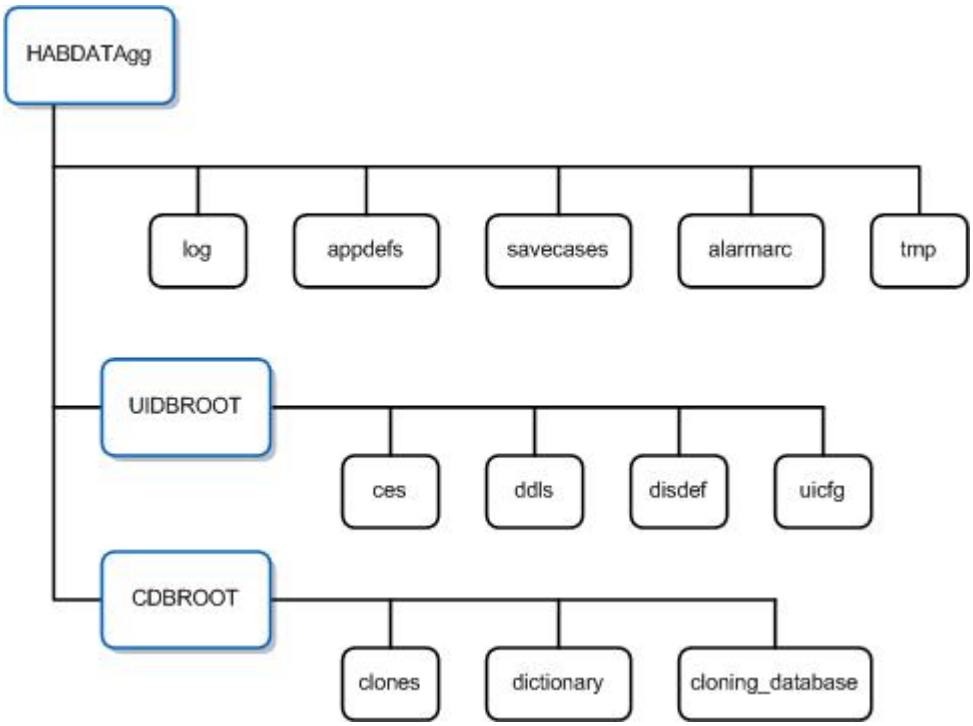

Figure 6. HABDATAgg Subtree

The HABDATAgg subtree contains all of the volatile data in the system, including savecases, log files, application definitions, temporary files, clone files, and display definition files.

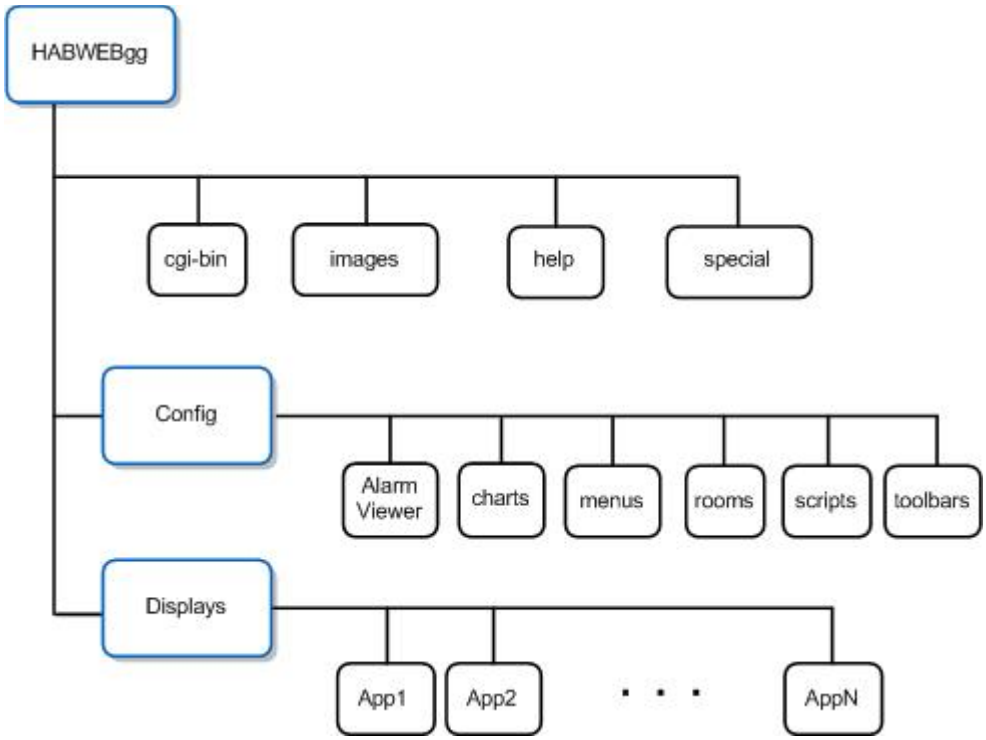

Figure 7. HABWEBgg Subtree

The HABWEBgg subtree contains all Browser displays and associated data within the system, including binary display files, Browser and Alarm Viewer configuration files, display bitmaps, chart configurations, toolbars, help files, and special Web pages.

2.3 Habitat Environment Variables

The Habitat environment variables allow product components and utilities to find required files without having direct knowledge of the physical layout of Habitat. Nearly all of the Habitat terminal (leaf) directories have an associated environment variable. The table below describes many of the environment variables and the locations to which they point.

Table 1. Habitat Environment Variables

Environment Variable	Physical Location	Contents
HABITAT_DIRROOT	/root	HABITAT root directory
HABITAT_BINDIR	/root/habitatgg/habitat/bin	HABITAT executables
HABITAT_APPDEFS	/root/habdatagg/appdefs	HABITAT app definitions
HABITAT_LIBDIR	/root/habitatgg/habitat/lib	HABITAT object libraries
HABITAT_INCDIR	/root/habitatgg/habitat/inc	HABITAT include files
HABITAT_DDLS	/root/habdatagg/uidbroot/ddls	HABITAT display files
HABITAT_CFGDIR	/root/habitatgg/habitat/cfg	HABITAT configuration files
HABUSER_BINDIR	/root/habitatgg/habuser/bin	HABUSER executables

Environment Variable	Physical Location	Contents
HABUSER_LIBDIR	/root/habitatgg/habuser/lib	HABUSER object libraries
HABUSER_INCDIR	/root/habitatgg/habuser/inc	HABUSER include files
HABUSER_CFGDIR	/root/habitatgg/habuser/cfg	HABUSER config files
HABITAT_MGRROOT	/root/habmgrgg	HABITAT manager root
HABITAT_MANAGER	/root/habmgrgg/habmgr	Customization files
HABITAT_MXDEFDIR	/root/habsrcgg/habuser/special/mxdefs	Hdb mxdef files
HABITAT_SOURCEROOT	/root/habsrcgg/habitat	HABITAT source root
HABITAT_SRCDIR	/root/habsrcgg/habitat	HABITAT codesets
HABUSER_SRCDIR	/root/habsrcgg/habuser	HABUSER codesets
HABITAT_LOGDIR	/root/habdatagg/log	Application log files
HABITAT_CASES	/root/habdatagg/savecases	Hdb savecase files
HABITAT_ALMARC	/root/habdatagg/almarc	Alarm archive files
HABITAT_TMPDIR	/root/habdatagg/tmp	Temporary files
HABITAT_CLONEROOT	/root/habdatagg/cdbroot	Root directory for clone files
HABITAT_CDBROOT	/root/habdatagg/cdbroot	Root directory for Hdb files
HABITAT_DATAROOT	/root/habdatagg	HABITAT data root
HABITAT_COREROOT	/root/habitatgg/habitat	HABITAT application root
HABITAT_CLONES	/root/habdatagg/cdbroot/clones	Clone files
HABITAT_UICFG	/root/habdatagg/uidbroot/uicfg	UI configuration files
HABITAT_WEBROOT	/root/habwebgg	Displays and other files used by Browser

2.4 File Protection and Ownership

All files installed by Habitat have a specific set of ownership and protection flags set. These protections prevent users in other groups from manipulating the files. It is critical that files retain the correct file ownership and protections when they are copied.

For Linux machines, all files should be owned by group hab*gg* and have an actual owner of hab*gg*, where *gg* is the Habitat group number. Complete access is granted to the owner and members of the group, whereas all other users have read and execute permissions only.

2.5 File Naming Conventions

Within Habitat, certain files and all directory trees can be identified by a naming convention. For files, those that begin with the HABMGR prefix can safely be modified by users. These files, wherever they may be found, will always be preserved through Habitat upgrades.

For directory trees, those that begin with HABITAT identify files owned by the Habitat system. Users should consider carefully before placing files in these directories, as they may not be preserved when Habitat is upgraded.

Users are guaranteed that directory trees with the HABUSER prefix will not be modified or removed during a Habitat upgrade.

2.6 Remote Users

The Habitat host system provides information to remote UI workstations through a network link from the Browser server to the clients.

2.7 Habitat in Windows

Habitat's relationship to the operating system involves more details for Windows than those described in the previous sections. In addition to accounts, directories, and environment variables, Habitat in Windows creates registry entries and may, optionally, be configured as a Windows service. For more information, refer to the *Habitat Software Installation and Maintenance Guide*.

3. Controlling and Configuring Habitat

This chapter describes, in detail, the procedures and functions that control Habitat during startup, login, and shutdown. This information is useful to both administrators and users, since both must frequently start up, shut down, or configure the environment in which Habitat operates.

All the files and procedures described in this chapter have similar names on the main platforms supported by Habitat: Windows and Linux. As explained, while the files themselves have similar names and identical purposes, the implementation and order of calling on the platforms are different. However, there is enough in common that anyone familiar with one platform should be able to easily migrate to the other.

3.1 Overview

This chapter deals with three separate, but interdependent, Habitat operations: how the login sequence proceeds, how Habitat starts, and how Habitat stops. In the sections below, each of these operations is outlined. Following those sections, detailed explanations of each sequence are provided, including how each may be customized. This chapter concludes with a discussion of how upgrades impact any customizations that have been made.

3.1.1 Login Sequence

The Habitat login sequence is the set of procedures that are executed when a user logs in to a Habitat account. The login sequence follows the logical flow described below:

1. The Habitat environment is obtained (explicitly through the execution of procedures in Linux or through the registry in Windows).
2. Habitat shortcuts are defined.
3. User customizations are performed.

3.1.2 Startup Sequence

The Habitat startup sequence is the series of procedures, executed either at boot time or manually by users, that starts the Habitat system, including defining all required environment variables and executing all tasks required to make Habitat operational.

Conceptually, the Habitat startup sequence is the same on all platforms:

1. Habitat environment variables are defined, including any user customizations.
2. Habitat core tasks are started.

In both Linux and Windows, environment variables are local to the process, so each user must explicitly obtain a copy of the environment at login time or via an initialization script prior to running the process.

3.1.3 Shutdown Sequence

This sequence is invoked when users or the system administrator need to stop all Habitat tasks. The most common reason for shutting down Habitat is to perform maintenance on system files.

Frequently, these files cannot be manipulated safely while Habitat is running. The following actions are performed when a user requests a Habitat shutdown:

1. Habitat user tasks are shut down.
2. Habitat core tasks are shut down.

3.2 Login/Startup/Shutdown Sequence

This section describes the specific procedures used during the login/startup/shutdown sequence on each platform.

> **Note:** The order in which procedures are executed on the different platforms varies, due to the ways that the different platforms handle environment variables.

3.2.1 Login Sequence on Linux

The purpose of the Linux login sequence is to establish the environment variables used by the different Habitat processes, and to set up command shortcuts (aliases). The procedures run during Linux login are diagrammed below. Application-specific environment variables should be defined in the habmgr_define_environment script.

Linux Login Process

```
HABITAT_MANAGER/habitat_login
  |
  +  > HABITAT_MANAGER/habmgr_define_environment
  |      |
  |      +  > HABITAT_MANAGER/habitat_environment_<host>_<group>
  |
  +  > ESCATOOLS/escatools_define_shortcuts
  |
  +  > HABITAT_BINDIR/habitat_define_shortcuts
  |      |
  |      +  > HABITAT_MANAGER/habmgr_define_shortcuts
  |
  +  > HABITAT_MANAGER/habmgr_login_announcements.txt
  |
  +  > HABITAT_MANAGER/habmgr_login
```

3.2.2 Startup Sequence on Linux

The Linux startup sequence is initiated via the “habitat start” command, and is illustrated in the diagram below. The habitat_start_tasks procedure starts only the Hdbserver task. Application-specific startup logic should be put in the habmgr_start_tasks script.

Linux Startup Process

```
HABITAT_BINDIR/habitat start
  |
  +  > HABITAT_BINDIR/habitat_start_tasks
         |
         +  > HABITAT_MANAGER/habmgr_start_tasks
```

3.2.3 Shutdown Sequence on Linux

The Linux shutdown sequence is initiated via the “habitat stop” command, and is illustrated below. The sequence synchronizes the shutdown of the system by first requesting any running PROCMAN instances to shut down via the PROCSTOP program; the shutdown sequence then ends when it verifies that the Hdbserver program has exited.

Linux Shutdown Process

```
HABITAT_BINDIR/habitat stop
  |
  +  > HABITAT_BINDIR/habitat_stop_tasks
```

3.2.4 Login Sequence on Windows

No specific login script is executed when logging in to Windows. All the environment variables necessary for Habitat processes are initialized from the registry. Command shortcuts are implemented as batch files in Windows.

3.2.5 Startup Sequence on Windows

Habitat can be configured to start as a service or not. The details of how to configure Habitat’s startup options in Windows can be found in the *Habitat Software Installation and Maintenance Guide*.

The HABITAT_MANAGER\habmgr_start_tasks.bat file is where application-specific startup tasks should be placed.

3.2.5.1 Startup Sequence on Windows When Not Running as a Service

When Habitat is not configured to start as a service, it is manually started from a command window by invoking the habitat.bat file with the start option. The .bat file in %ESCATOOLS% calls the HABITAT_BINDIR\habitat.pl Perl script, passing the start option. The Perl script first starts hdbserver and then executes the HABITAT_MANAGER\habmgr_start_tasks.bat file to handle the remainder of the startup.

The HABITAT_MANAGER\habmgr_start_tasks.bat file is where application-specific startup tasks should be placed.

3.2.5.2 Startup Sequence on Windows When Running as a Service

When Habitat is configured to start as a service, the operating system runs the HABITAT_BINDIR\tview_server.exe task and passes it the HABITAT_MANGER\habitat_service_env.cfg configuration file that contains the environment. The Habitat service normally runs using Network Service credentials (the principle of using the least privileged account possible), so it does not have direct access to the registry settings for the Habitat account.

The tview_server task calls the habmgr_start_tasks.bat routine in the HABITAT_MANAGER folder to handle the remainder of the startup.

The HABITAT_MANAGER\habmgr_start_tasks.bat file is where application-specific startup tasks should be placed.

3.2.6 Shutdown Sequence on Windows

The Windows shutdown sequence is also initiated via the “habitat stop” command. The sequence synchronizes the shutdown of the system by first requesting any running PROCMAN instances to shut down via the PROCSTOP program; the shutdown sequence then ends when it verifies that the Hdbserver program has exited.

3.3 Customizing Your Habitat Environment

This section discusses the customizations that are common to the platforms. For further platform-specific customization information, refer to the later sections of this chapter.

Three areas of the Habitat environment can be customized:

- Tasks run at Habitat startup
- Environment variables defined at startup
- Actions taken at login time

The sections that follow detail the procedures for customizing these three areas.

3.3.1 Customizing Startup Tasks

All platforms allow users to customize, in the same way, which tasks are started when Habitat is started. Users can place in the HABITAT_MANAGER logical directory a file called “habmgr_start_tasks” (with a “.com”, “.sh”, or “.bat” extension depending on the platform). This procedure is executed by the startup procedure as described above. It may contain instructions to start whatever tasks users desire.

Users are guaranteed that, prior to invoking the habmgr_start_tasks function, the habitat_start_tasks procedure has started the HdbServer task. The habitat_start_tasks procedure contains synchronization functions to ensure that HdbServer is running. Typically, a user starts the PROCMAN model appropriate to this Habitat in the habmgr_start_tasks procedure. For example, on Linux:

```
#!/bin/ksh
#
# Procedure: habmgr_start_tasks.sh
# Purpose: Start user tasks at startup
#
tview run procman procman ems
#
exit
```

> **Note:** While no limit is enforced, you should refrain from placing non–Habitat “startup” commands in these files. These files are preserved through Habitat upgrades.

3.3.2 Customizing the Habitat Environment Variables

Two types of customization can be performed on environment variables used by Habitat:

- Addition of user environment variables

- Modification of Habitat environment variables

3.3.2.1 Adding User Environment Variables

For Linux, you can define environment variables in the habmgr_define_environment file in the HABITAT_MANAGER directory, then use the “habitat update” command to validate the newly created environment variable. It is also necessary to log out and log back in to trigger the login process to source the new environment settings.

For Windows, one way to add a user environment variable is as follows:

1. Right-click on the My Computer icon on the desktop.
2. Select the Advanced tab in the System Properties dialog box.
3. Click the Environment Variables button to open the environment variable editing dialog box.
4. Add the desired environment variable under the User Variables section.

You also must log out and log back in to make the new environment variable effective.

If Habitat is configured as a Windows service, a reboot is necessary to make the environment variable visible to the Habitat service.

3.3.2.2 Modifying Habitat Environment Variables

When a user wishes to modify an environment variable owned by Habitat, the procedure used is identical to adding a user-owned value. The difference lies in how Habitat processes this definition.

For Linux, the environment variable values owned by Habitat are validated by a special procedure, habitat_define_environment, located in HABITAT_BINDIR. This procedure runs when the “habitat update” command is issued. If the environment variable value cannot be validated (because it points to a directory or file that does not exist), a warning is issues, and corrections must be made before Habitat can be started or before the environment file is modified.

In some cases, the habitat_define_environment procedure may determine and set a default value for user-customized Habitat–owned environment variables that cannot be validated. If a value is defaulted, habitat_define_environment generates a warning message.

For Windows, the Habitat environment variables must be modified using the steps described in the previous section. At this time, there is no procedure to validate the changed Habitat environment variables.

For example, to customize the location of HABUSER_LIBDIR, the following steps are required:

1. Shut down Habitat.
2. Move the HABUSER_LIBDIR files to their new location.

   File ownership and protection must be set correctly.
3. Modify the file HABITAT_MANAGER/habmgr_define_environment to contain the new definition of HABUSER_LIBDIR.

For Linux:

1. Issue the “habitat update” command.
2. Verify that the Habitat environment variables are correctly defined.

3.3.3 Customizing the Login Procedure

For Linux, Habitat provides numerous opportunities for the system administrator to customize the login environment for users. An outline of the procedures provided and their intended purpose is provided below. The procedures are presented in order of execution during the login process.

- habmgr_define_environment: Refer to the previous section for a description of the purpose of this file.
- habmgr_define_shortcuts: This procedure is designed to allow the administrator to define whatever shortcuts (aliases or symbols) make user interaction easier.

  > **Note:** Habitat command shortcuts are defined in the habitat_define_shortcuts procedure, invoked prior to this function.

- habmgr_login_announcement: Contains any text that the administrator wishes to be output to users' terminals when they log on. This file should contain text only, as it is sent to stdout by the login procedure.
- habmgr_login: This procedure allows the system administrator to perform any other login activities. This procedure is used to invoke login functions for layered products or environment-specific functions.

For Windows, there is currently no provision for customizing the login procedure.

3.4 Linux-Specific Issues

This section describes features of the startup/login/shutdown sequence that are unique to Linux.

Environment Variable Definition File

Since Linux does not have the concept of environment variables that persist beyond the life of the process, environment variables in Linux are defined during the login process. To attempt to maintain a consistent environment between users, specific action must be taken to update the Habitat–owned environment variables. By issuing the "habitat update" command, a user causes Habitat to re-evaluate the definitions of Habitat environment variables based on the current user environment.

For example, suppose a user wishes to move the HABITAT_LOGDIR directory to another part of the file system. The following steps are required:

1. Create the new directory and copy the required files.
2. Redefine the affected environment variables in habmgr_define_environment.sh

   ```
   % setenv HABITAT_LOGDIR /newloc/newdir/log
   ```

3. Issue the command for Habitat to regenerate the habitat_environment_*hostname_group*.sh file.

   ```
   habitat update
   ```

   > **Note:** When this command is issued, a new shell is spawned, so only definitions contained in the habmgr_define_environment file are picked up. Any local customizations to the environment do not influence the generation of the new Habitat environment file.

4. Log out and log back in to obtain the new environment.

> **Note:** The only time that Habitat attempts to validate environment variables is when the update procedure is executed. No login-time checks are made of the validity of entries that the user places in habmgr_define_environment, and no customizations of the environment are placed online until the update procedure is executed.

3.5 Windows-Specific Issues

For Windows, no login customization is available when Habitat is started from a command window; all environment variables are defined in the registry. These variables must be edited using the Environment Variables dialog box.

When Habitat is running as a service, the environment is loaded from the HABITAT_MANAGER\habitat_service_env.cfg file. This file can be edited directly, however the preferred procedure is documented in the *Habitat Software Installation and Maintenance Guide*.

You can customize the startup tasks by editing the habmgr_start_tasks.bat file, which is used by the "habitat" program and the Habitat as service startup program.

3.6 Preserving Your Changes through Upgrades

Normally, upgrading a Habitat installation does not impact any customizations to the habmgr_define_environment file. However, on some occasions, GE must add or make changes to an environment variable. In these cases, the *Habitat Release Notes* should provide detailed information about what has changed and the impact it has on customizations. Otherwise, no action is needed, and any customizations made to the HABMGR files are preserved through upgrades. However, it is recommended that customers track all customizations to the HABMGR files by storing these in a source code management system.

4. Habitat Operations

This chapter describes many frequently used Habitat commands. It provides an overview of the most common operations and the tools used to perform them. By reading the remaining sections of this chapter, you will learn to:

- Start up and shut down Habitat.
- Set application and family context.
- Navigate through Habitat directories.
- Monitor and control Habitat tasks.
- View log databases.
- Manage HABITAT clones and savecases.

For more information about ESCATOOLS, refer to the *ESCATOOLS User GuideUser Guide*.

For more information about Hdb, refer to the *Hdb User Guide* or the *Hdb User Guide*.

For more information about working with Browser, refer to the *Browser Software Installation and Maintenance Guide*.

4.1 Habitat Startup and Shutdown

Habitat can be started or shut down by any user logged into the hab*gg* account.

> **Note:** Before shutdown, it is essential that all users be notified, since all applications running within Habitat are stopped.

4.1.1 Starting Habitat

Normally, Habitat is automatically started at boot time via the procedures described in the *Habitat Software Installation and Maintenance Guide*. However, there may be other occasions when Habitat must be started or stopped.

Once Habitat has been started on a given system, it can be restarted with the command:

```
habitat start
```

4.1.2 Stopping Habitat

Shutting down a Habitat system causes all applications running in the Habitat environment to shut down as well.

Shut down Habitat by issuing the command:

```
habitat stop
```

The “stop” command attempts to perform a controlled shutdown of any PROCMAN instances found. Applications not under PROCMAN control are terminated (killed) immediately after all running PROCMAN instances are shut down.

Note: This can result in the loss of data.

4.2 Setting the Application and Family Context

Many Habitat commands require an application and family context to be set before they can be executed. This context defines which clone is affected by the command. When working from a command window (DECterm, Xterm, or Command Prompt), you can set the context with the command:

context *application family*

where *application* and *family* identify the specific clone to be referenced.

No check is performed that the context represents a valid clone. The current context can be shown by issuing the command with no application and family information. In Linux, users are given the opportunity to change the context at that time.

4.3 Navigating Through Habitat Directories

Nearly all the directories in Habitat that contain actual files, or to which a user may wish to navigate, have an associated environment variable. These environment variables can be used along with standard operating system commands to navigate to various directories within Habitat. For example, to change the default directory to HABITAT_BINDIR, the commands used are:

Linux:

```
% cd $HABITAT_BINDIR
```

Windows:

```
> cd /d %habitat_bindir%
```

For Linux, the command line is case-sensitive. For Windows, the "/d" qualifier is used to change drive letters in addition to directories so that, if HABITAT_BINDIR is located on a different drive than the current default, the default drive is changed as well.

There is one exception to the practice of having an environment variable for all terminal directories. There are no environment variables defined for the subsystems and codesets in the HABITAT and HABUSER source code trees. Instead, a special "getenv" command is delivered by Habitat, to assist in navigating to the source code directories. The syntax for the command is:

getenv *codeset*

where *codeset* is the name of a codeset in either the HABUSER_SRCDIR or the HABITAT_SRCDIR directory structure. This command sets the default directory to the codeset specified, if there is one by that name. Otherwise, this command considers the name to be that of a subsystem and it sets the default directory to the corresponding subsystem directory, if there is one.

In addition to environment variables and commands for navigation, Habitat modifies the definition of the PATH environment variable in Linux and Windows to contain the HABUSER_BINDIR and HABITAT_BINDIR directories. This allows a user to run programs in these directories without specifying their location.

4.4 Monitoring and Controlling Habitat Tasks

The UI provides most of the information required to monitor and control Habitat tasks. However, there are occasions when the UI is either not available or not convenient. For these cases, Habitat provides a number of command-line tools. The two most important tools are TVIEW (task viewer) and LISTMSG (list message).

4.4.1 Controlling Tasks with TVIEW

Use the TVIEW utility to start, stop, and view the tasks running on Habitat, interactively. TVIEW is invoked with the command:

tview

TVIEW operates like other Habitat menu-driven tools. To issue a command, type the highlighted letter of the command or its name, followed by any combination of the qualifiers listed next to the command.

```
             Task Viewer
             -----------
   LIST        [/FULL /TASK /TIME /APP /FAM /SUB /HB]
   RUN
   STOP        [/TASKID /FAMILY /FORCE /ALL]

   Help
   Exit

tview> :
```

Alternately, TVIEW can be used in a command-line, non-interactively, where you specify the command to be executed when you invoke TVIEW. For example:

tview run procman procman ems

However, once at the TVIEW prompt, you can specify values only when prompted for them. Typing the “help” command redisplays the menu choices.

4.4.1.1 The List Command

The most frequently used command is the “list” command; it displays a summary of all the tasks currently running under Habitat.

By default, this list is displayed in alphabetical order by task name. The /FULL qualifier can be used to obtain more detail regarding each task; the other qualifiers are used to order or filter the display as follows:

- /TASK: Ordered by task ID
- /TIME: Ordered by start time
- /APP: Ordered by application name
- /FAM: Ordered by family name
- /SUB: Filtered by task name substring
- /HB: Show heartbeat time and its counter of Habitat applications.

> **Note:** Only one of the filters can be used at a time. The filters can be combined with the /FULL qualifier.

4.4.1.2 The Run Command

The task viewer can be used to start Habitat tasks with the TVIEW "run" command. To start a task this way, the task must have an application definition file loaded into the HABITAT_APPDEFS directory.

To run a task from TVIEW, issue the "run" command and fill in the values for task name, application, and family when prompted.

The following example shows the PROCMAN task being started in family EMS:

```
              Task Viewer
              -----------
   LIST         [/FULL /TASK /TIME /APP /FAM /SUB /HB]
   RUN
   STOP         [/TASKID /FAMILY /FORCE /ALL]

   Help
   Exit

tview> : run
 [RUN] Enter the Task Name []:procman
 [RUN] Enter the Application Name []:procman
 [RUN] Enter the Family Name []:ems
 [RUN] Enter Parameter String (if any) []:
APPMGR_S_NORMAL, normal successful completion
```

For the example to work correctly, there must be an application definition (APPDEF) file called "PROCMAN" in the HABITAT_APPDEFS directory that contains the definition for a task called "PROCMAN".

4.4.1.3 The Stop Command

Shutting down a task with TVIEW is accomplished using the "stop" command. You can specify which task to shut down in one of several ways:

- STOP: Prompts for the task name that should be stopped. You only need to enter enough of the beginning of the name to create a unique substring.
- STOP/TASKID: Prompts for the APPMGR task ID (not the PID) of the task to shut down.
- STOP/PID: Prompts for the task PID.
- STOP/FAMILY: Prompts for a family name and shuts down all tasks running under that family.
- STOP/ALL: Stops all tasks running in Habitat.

Sometimes, despite a proper request, a task fails to stop. In these cases, you can use the /FORCE qualifier to cause TVIEW to take more drastic action to stop the task, similar to "kill -9" in Linux. The /FORCE qualifier can be combined with any of the task-selection qualifiers described above.

```
            Task Viewer
            -----------
   LIST             [/FULL /TASK /TIME /APP /FAM /SUB /HB]
   RUN
   STOP             [/TASKID /FAMILY /FORCE /ALL]

   Help
   Exit

tview> : stop
 [STOP] Enter the Run-time Task Name []:procman_procman_ems
APPMGR_S_NORMAL, normal successful completion
```

4.4.1.4 Running Tasks Interactively

In addition to starting tasks with TVIEW, you can choose to run a task directly at the command prompt. **Important:** When running a task interactively, the correct context must be set using the "context" command.

Depending on the specific application code, the task may or may not appear in the list of tasks in TVIEW.

Running tasks interactively is usually done only for debugging purposes.

4.4.2 Viewing Message Logs with LISTMSG

Habitat includes a command-line utility called LISTMSG for viewing the contents of message logs within clones.

The syntax of the listmsg command is:

listmsg *dbname*

The utility supports several optional qualifiers:

- **a** or **app:** Used to specify the name of the application containing the log database, overriding the current default context.
- **f** or **fam:** Used to specify the family name, overriding the current default context.
- **record:** Used to specify a non-standard record name for the logging database (the default record name is "mess").
- **since:** Can be used to restrict the output to records that have been added since the specified time. A complete time qualifier in the format "dd:mm:yyyy:hh:mm:ss" must be specified.
- **reverse:** Causes the utility to output records in chronological order, from earliest to latest. By default, LISTMSG shows records in reverse chronological order (from latest to earliest).
- **output:** Redirects the output to the specified file name.

Note that these qualifiers are specified using a "-" in Linux and Windows.

The HdbServer task must be running for the utility to gain access to the log database.

An example of the listmsg command in Linux is:

```
% listmsg proclog -a procman -f ems -reverse
```

```
% listmsg proclog -app=procman -fam=ems -reverse
```

4.4.3 Running Tasks and Paths from the Command Line

A tool named proccmd is delivered with Habitat. This tool can be used from the command line or from a script file, to signal PROCMAN to start or stop a task or to run a path. Although TVIEW can also start and stop tasks, TVIEW has the disadvantage that the task is not under PROCMAN's control and thus retries are not done if the task fails.

The syntax of proccmd is very simple:

```
proccmd starttask <task name> <procdbclerk family>
proccmd stoptask <task name> <procdbclerk family>
proccmd runpath <path name> <procdbclerk family>
```

The PROCDBCLERK family is required so that, if multiple PROCMANs are being run (as is the case when a DTS is running), proccmd knows which PROCDBCLERK to send the command to.

Normally, there is no response to the proccmd launch unless the PROCDBCLERK referenced is not running.

> **Caveat:** If multiple invocations of proccmd are done simultaneously, errors could occur. This is because proccmd assigns itself a client name in order to pass the RUSER commands to PROCDBCLERK. If two copies of proccmd try to run at the same time, the second fails trying to register the same client name.

4.5 Managing Clones and Savecases

A key concept in Habitat is understanding how to manage clones and the data they contain. This section describes the three utilities used to manage clones and their data (savecases). These utilities are:

- HdbCloner: Used to create, update, and remove clones
- HdbCopydata: Used to retrieve and create savecase data
- HdbRio: Used to access the data in an online clone or an archive

The following sections detail the usage of each tool.

4.5.1 The HdbCloner Utility

This utility can be used to create, remove, and update clones. The HdbCloner utility can also be used to load schema definitions into Habitat.

The basic format for issuing a command with HdbCloner is as follows:

```
hdbcloner -c command -a application -f family [options]
```

The application and family qualifiers must be used, although in some cases a wildcard value ("*") can be specified. The detailed instructions for each of the following commands are contained in the next few sections.

- show_clone: Displays information about a clone

- create_clone: Creates a new clone
- remove_clone: Removes an existing clone

Many HdbCloner commands have online help available. To see general information, issue the command:

```
hdbcloner -h
```

For help with a specific command, append the "-h" qualifier after the command:

```
hdbcloner -c show_clone -h
```

4.5.1.1 Displaying Clone Information

By specifying "show_clone" as the HdbCloner command, you can see information about existing clones. This command supports the "*" wildcard character for both the application and family name. An optional "-full" qualifier can be used with this command.

```
hdbcloner -c show_clone -a procman -f ems -full

Clone: PROCMAN.EMS               *ON-LINE*     Schema: PROCMAN
       Clone Unique ID: 75
       Attributes: NOT-REPLICATED
       Segment Alignment: 8192      ALPHA PAGE
       File: DKB0:[ESCA.HABDATA60.CDBROOT.CLONES]clone_procman_ems.car

       DB: PROCMAN     Size: 165910
           GUID: 75,1,0,0
           Definition Stamp: 453155060,0     0x1B0298F4,0x00000000
       DB: PROCLOG     Size: 84140
           GUID: 75,2,0,0
           Definition Stamp: 515662,0     0x0007DE4E,0x00000000
Cloner completed successfully
```

4.5.1.2 Creating a New Clone

In order to create a clone, the required schema files must first be loaded into Habitat. Once all the schemas are available, a clone can be created by using the "create_clone" command option. The optional components of the "create_clone" command include:

- -d: Specifies a directory path for where to place the clone file. Use this qualifier to place clones in a directory other than the default HABITAT_CLONES directory. This is frequently done for performance reasons.
- -replicate: Indicates that the clone should be created with the "replicate" flag. A clone must be created with this option for the replication subsystem (MRS) to access the data in the clone.

For a discussion of the other qualifiers supported during clone creation, refer to the *Hdb User Guide*.

Below is an example of creating a replicated version of the Alarm clone:

```
hdbcloner -c create_clone -a alarm -f ems -replicate
Clone file created: DKB0:[CLONES]clone_alarm_ems.car
Clone ALARM in family EMS is ON-LINE
Clone ALARM in family EMS  *CREATED*
Cloner completed successfully
```

4.5.1.3 Removing an Existing Clone

The HDBCLONER utility can be used to remove a previously installed clone. The command option used for this operation is “remove_clone”.

```
hdbcloner -c remove_clone -a alarm -f ems
Clone ALARM in family EMS is OFFLINE
Clone file deleted: DKB0:[CLONES]clone_alarm_ems.car
Clone ALARM in family EMS  *REMOVED*
Cloner completed successfully
```

> **Note:** Even if a clone has been created in an alternate clone directory, there is no need to specify the alternate path when removing the clone.

4.5.1.4 Updating a Clone

Updating a clone uses the same process as creating one, but the “-replace” qualifier is added. At the same time, other qualifiers can be added to change a clone’s attributes — for example, “-replicate” or “-noreplicate”.

```
hdbcloner -c create_clone -a procman -f ems -replace
Setting clone PROCMAN.EMS OFFLINE
Clone temporary file created using family: __124
Clone data copied to temporary clone file
Deleted old clone file: DKB0:[CLONES]clone_procman_ems.car
Clone temporary file renamed to clone file:DKB0:[CLONES]clone_procman_ems.car
Clone PROCMAN in family EMS is ONLINE
Clone PROCMAN in family EMS  *REPLACED*
Cloner completed successfully
```

An additional qualifier, -nocopy, can be specified, indicating that the data in the clone should not be preserved when the clone is replaced.

4.5.2 The HdbCopydata Utility

The data that resides within clones is managed primarily through archives and savecases. Both are snapshots of the data in one or more of the databases that make up the clone.

The name archive can be used to generically describe any snapshot, whether archive or savecase. A savecase generally denotes a specific type of snapshot (the rest of this chapter uses the generic term “archive”).

Habitat provides a utility, HdbCopydata, for creating and retrieving these files. The operation of this utility is much like the HdbCloner utility. The general format for HdbCopydata archive commands is:

```
hdbcopydata -sf (-df) filename -s (-d) application.family
```

where:

- -sf: Represents a source file name, indicating an archive retrieval.
- -df: Represents a destination file name, indicating an archive creation.
- -s: Specifies the source clone, used in archive/savecase creation.
- -d: Specifies the destination clone, used in archive/savecase retrieval.

Unlike the HdbCloner utility, the application and family are specified after one qualifier with a "**.**" between the names.

For "savecase" commands, the format is slightly different:

```
hdbcopydata -case type -title title -[s,d] application.family
```

where:

- -case: Indicates that the operation is on a savecase of the specified type.
- -title: Specifies the title portion of the filename. (Refer to the following section for how savecase filenames are formed.)

4.5.2.1 Savecases vs. Archives

Two terms are frequently used to describe snapshots of data made with the HdbCopydata utility: "savecase" and "archive". The difference between the two is minor.

A "savecase" is a special kind of archive file with a pre-defined set of databases or partitions of databases. The savecase definition is stored inside the clone schema (CLS) files.

Savecases also differ in the name of the file that is created. The first half of the name is defaulted to "case_*application_casetype*". The only portion of the name that you specify is called the "title". The full savecase name is "case_*application_casetype.title*".

An additional difference is that savecase operations create and retrieve files from the HABITAT_CASES directory, while archive operations use the default directory — unless the directory is specified as part of the file name.

To create or retrieve a savecase instead of an archive file, use the "-case" qualifier with the "hdbcopydata" command. Savecases are recognized by the Savecase Manager program (see below) and they are displayed in the Savecase Manager displays, whereas archives are not recognized by the Savecase Manager so they are not shown in the Savecase Manager displays.

4.5.2.2 Creating Savecases and Archives

The process of creating a savecase starts with selecting the portion of data from the clone that is to be stored in the archive (or savecase). Once this is decided, the "hdbcopydata" command is issued.

The following two commands illustrate the creation of a savecase and an archive:

```
hdbcopydata -case alarm -title esca60 -s alarm.ems
Copying data from: CLONE  to: SAVE CASE
   Saving database: ALARM.ALARM.EMS
Copydata completed successfully

hdbcopydata -df case_alarm_alarm.esca60 -s alarm.alarm.ems
Copying data from: CLONE  to: ARCHIVE FILE
   Archiving database: ALARM.ALARM.EMS
Copydata completed successfully
```

To perform archive operations on specified databases (even partitions), pre-pend the database name to the clone name in the format "*partition.database.application.family*".

For the first command, the savecase is located in HABITAT_CASES; for the second command, the archive file is created in the current default directory.

While both commands create a file named “case_alarm_alarm.esca60”, the contents of the two files could be quite different depending on how the savecase definition “alarm” is defined in the clone schema file (CLS). If the “alarm” savecase definition in the CLS file includes the alarm database only, then the savecase and the archive contain the same data after they are created. If the “alarm” savecase definition contains only a subset of the partition in the alarm database or if it includes other databases defined in the clone schema, then the two generated files are quite different internally.

The recommended practice is to not use the savecase file naming convention when naming an archive file, to avoid possible confusion.

Note that, when creating an archive, if no database information is specified with the “-s” qualifier, all the databases in the clone are saved into the archive. For example:

```
hdbcopydata -df case_alarm_entire.esca60 -s alarm.ems
Copying data from: CLONE  to: ARCHIVE FILE
   Archiving database: ALARM.ALARM.EMS
   Archiving database: ALARMLST.ALARM.EMS
   Archiving database: ALARMESS.ALARM.EMS
   Archiving database: ALARMHIS.ALARM.EMS
   Archiving database: ALARMTST.ALARM.EMS
Copydata completed successfully
```

4.5.2.3 Retrieving Savecases and Archives

The process for retrieving an archive is similar to the create operation. The following operation retrieves the savecase that was created in the preceding section:

```
hdbcopydata -d alarm.ems -sf case_alarm_alarm.esca60
Copying data from: ARCHIVE FILE  to: CLONE
   Copying database: ALARM.ALARM.EMS
   Skipping database: ALARMLST(Not found in source)
   Skipping database: ALARMESS(Not found in source)
   Skipping database: ALARMHIS(Not found in source)
   Skipping database: ALARMTST(Not found in source)
Copydata completed successfully
```

If a savecase is to be retrieved, you can either specify a pathname with the filename (HABITAT_CASES), or use the “-case” and “-title” options, as follows:

```
hdbcopydata -d alarm.ems -case alarm -title esca60
```

Note that, when the “-case” qualifier is used, HdbCopydata automatically assumes that the case is located in HABITAT_CASES.

4.5.2.4 Displaying Available Savecase Types

The HdbCloner utility can be used to examine the clone schema file, to obtain information about what types of savecases are supported by an application. Issue the following command:

```
hdbcloner -c show_schema -n cls:application -full
```

An example of the commands output is show below:

```
hdbcloner -c show_schema -n cls:alarm -full

CLS Schema ALARM
```

```
    DB: ALARM.ESCA_HABITAT
    DB: ALARMLST.ESCA_HABITAT
    DB: ALARMESS.ESCA_HABITAT
    DB: ALARMHIS.ESCA_HABITAT
    DB: ALARMTST.ESCA_HABITAT

    Case: ALARM
       Includes Database ALARM
    Case: ALARMLST
       Includes Database ALARMLST
    Case: ENTIRE
       Includes Database ALARM
       Includes Database ALARMESS
       Includes Database ALARMHIS
       Includes Database ALARMLST

Cloner completed successfully
```

4.5.2.5 Managing Savecases from the UI

In addition to the command-line utility HdbCopydata, savecases can be manipulated via Browser.

A single application permits the retrieval and creation of savecases for all eligible applications in the system. This application is called the “Savecase Manager” or “Casesrv”. The master display for this application is available from the Browser display directory, or from the HABITAT Applications menu.

Start by accessing the Savecase Manager Master display via any method. A display showing all the supported applications on the system appears, as shown in Figure 8. To choose which application you wish to manipulate savecases for, select the application.

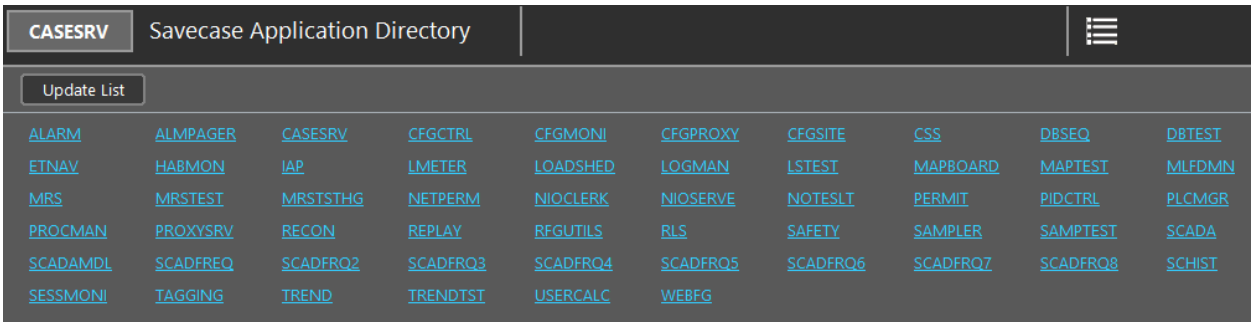

Figure 8. Savecase Master Display

Once you choose an application, the next display (shown in Figure 9) lists the available savecases for that application, ordered by TYPE and TITLE. To retrieve, delete, or replace an existing case, right-click on the savecase title. From the menu options presented, choose “Retrieve…”, “Update…”, or “Delete…”. For each choice, a pop-up showing the savecase complete definition (Application, Type, and Title) asks you to confirm the operation. Choosing “Retrieve…” causes this savecase to be loaded into the clone.

Note that the current application family assignment in PERMIT controls which family of the application the data is loaded into. Choosing “Update…” causes the existing savecase to be replaced with the data currently in the clone. The “Delete…” option is used to remove the savecase from the system.

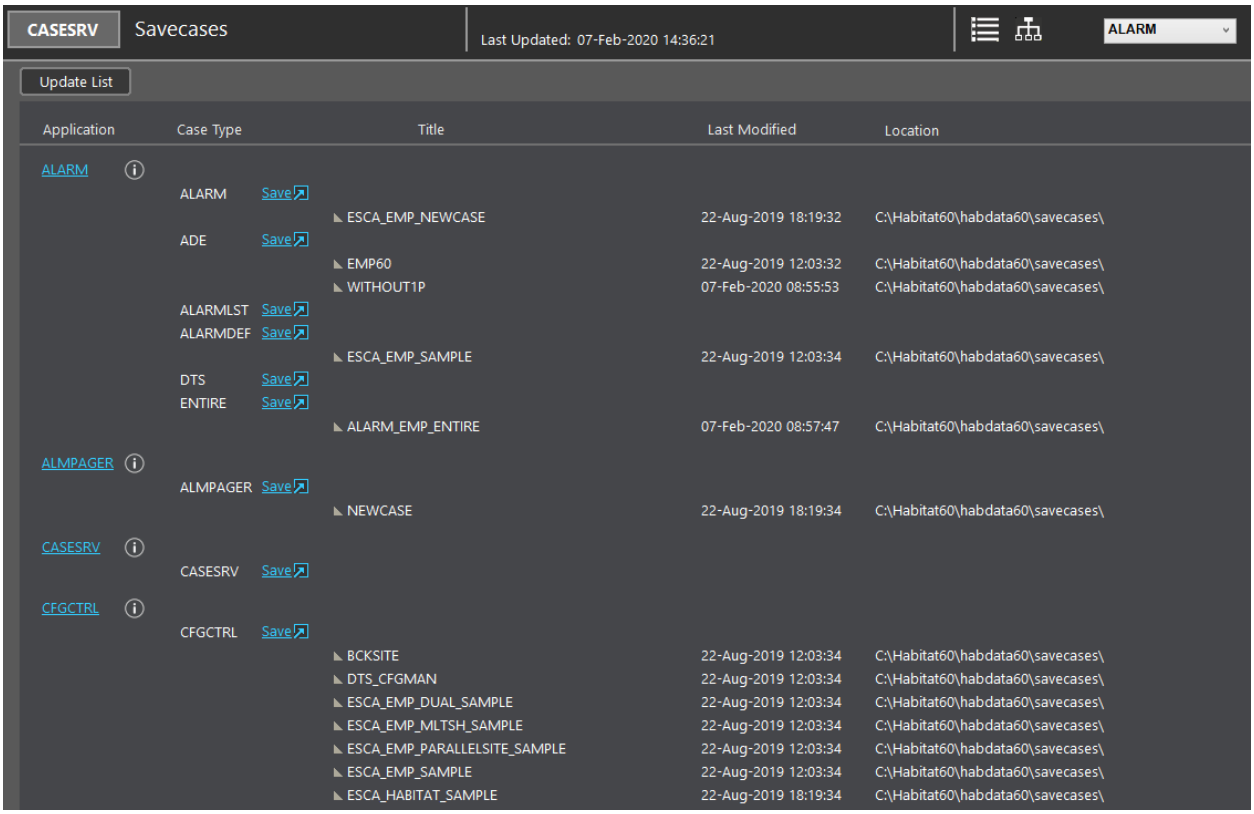

Figure 9. Savecase Application Display

To create a new savecase, type the desired title of the new savecase on the command line that is part of the viewport, and then click the Save button next to the case type desired.

4.5.3 The HdbRio Utility

Once a clone has been created, its databases can be accessed by the HdbRio tool. This tool allows a user to view the actual fields inside the database and modify their contents. In addition, records within the database can be inserted, deleted, or copied.

The command to invoke HdbRio is shown below:

```
hdbrio [-a application -f family -archive archive] database
```

If no application and family names are specified on the command line, the current default context is used instead. If the "-archive" qualifier is used, then HdbRio accesses an archive file instead of a clone.

The next few sections describe some basic HdbRio functions. These include:

- Determining what records are in the database
- Navigating through records
- Reading and modifying record fields
- Inserting and deleting records

HdbRio provides much more functionality than is described in this document. For more information, refer to the *HDB User Guide*, or obtain help from the HdbRio tool itself by issuing the "help" command

from within HdbRio; for more-detailed help about specific commands, type “help *command*” at the HdbRio prompt.

4.5.3.1 Examine the Records in a Database

After invoking HdbRio to gain access to a database, the “list” command can be used to examine the types of records contained in the database and how many of each record type are present.

```
hdbrio -a nioserve -f ems niosrvnv
[NIOSRVNV.NIOSERVE.EMS]
rio>list
   Legend:
   =======
     NH = Non-hierarchical      H = Hierarchical
      I = ITEMS                 M = Maxlv
      C = Circular              F = Free
     SZ = Record size in bytes (with pointers and alignment added)
     MX = mx of record type
     LV = lv of record type
     U% = Percentage used (MAXLV,CIRCULAR,ITEMS not included)
     UB = Estimated bytes not used by record type (i.e. no data)

   Record  Summary  of  database  NIOSRVNV
   ========================================
      ALIAS  LV=    0  MX=  100   H  SZ= 116  U%=  0%  UB=  11600
      CLERK  LV=    5  MX=   72   H  SZ=  84  U%=  7%  UB=   5628
      ITEMS  LV=    1  MX=    1   I  SZ= 112
     NVDECK  LV=    2  MX=    2   H  SZ=  24  U%=100%  UB=      0
     SERVER  LV=    5  MX=   48   H  SZ=  84  U%= 10%  UB=   3612
       SITE  LV=    3  MX=   36   H  SZ=  32  U%=  8%  UB=   1056
     SYSTEM  LV=    4  MX=   48   H  SZ=  20  U%=  8%  UB=    880
       TEXT  LV=    0  MX=  200   F  SZ=  92  U%=  0%  UB=  18400

     Number of record types = 8
[NIOSRVNV.NIOSERVE.EMS]
rio>
```

The display shows the records contained in the NIOSRVNV database of the NIOSERVE.EMS clone. For each record type, the display shows the current number of records (LV) and the maximum number (MX) that can be in the database. Also shown is the type of record: hierarchical (H), non-hierarchical (NH), or free (F).

The SZ value shows the size of the record type in bytes, including the pointer fields and alignment padding added. The U% value shows the percentage of utilization for the record type. The UB value shows the estimated total number of bytes not currently filled with data for the record type.

4.5.3.2 Navigating in HDBRIO

There are several ways to navigate around in HDBRIO, including:

- Positioning: Use these functions to jump to a specific record subscript.
- Finding: Use these functions to find a specific record by searching for its ID.

- Moving up/down the hierarchy: Use these functions to browse the record hierarchy, moving up and down through parent and child records.

Positioning is accomplished with the “position” command. The record name and subscript must also be specified, as shown below:

rio> position *recordname = subscript*

To search for a record with the “find” command, you must know two things: the key field of the record and the key value desired. The key field for a record is designated in the database definition file (DBDEF). The command uses the following syntax:

rio> find *recordname = keyname*

The final method of navigation is moving up and down the hierarchy. The user must first be positioned (either through find or position) on a record in the hierarchy. From there, use the “up” and “down” commands.

4.5.3.3 Reading and Modifying Fields

Once the user is positioned on the desired record, the field values in the record can be read and/or modified. To examine the contents of all fields in a record, issue the “read” command.

The example below shows the use of several “hdbrio” commands.

```
hdbrio -a nioserve -f ems niosrvnv
[NIOSRVNV.NIOSERVE.EMS]
rio> list
      Legend:
      =======
        NH = Non-hierarchical      H = Hierarchical
         I = ITEMS                 M = Maxlv
         C = Circular              F = Free
        SZ = Record size in bytes (with pointers and alignment added)
        MX = mx of record type
        LV = lv of record type
        U% = Percentage used (MAXLV,CIRCULAR,ITEMS not included)
        UB = Estimated bytes not used by record type (i.e. no data)

      Record  Summary  of  database  NIOSRVNV
      ========================================
         ALIAS  LV=     0  MX=   100   H  SZ= 116  U%=  0%  UB=   11600
         CLERK  LV=     5  MX=    72   H  SZ=  84  U%=  7%  UB=    5628
         ITEMS  LV=     1  MX=     1   I  SZ= 112
        NVDECK  LV=     2  MX=     2   H  SZ=  24  U%=100%  UB=       0
        SERVER  LV=     5  MX=    48   H  SZ=  84  U%= 10%  UB=    3612
          SITE  LV=     3  MX=    36   H  SZ=  32  U%=  8%  UB=    1056
        SYSTEM  LV=     4  MX=    48   H  SZ=  20  U%=  8%  UB=     880
          TEXT  LV=     0  MX=   200   F  SZ=  92  U%=  0%  UB=   18400

      Number of record types = 8

[NIOSRVNV.NIOSERVE.EMS]
rio> position server = 2

[NIOSRVNV.NIOSERVE.EMS]
```

```
SERVER(2)> read
                    Display of record SERVER(2)
      ==========================================
       Integer*4    P__SITE_SERVER = 1
         Char*64     IPHOST_SERVER = ponti.esca.com
          Char*2      GROUP_SERVER = 32
          Char*6       NODE_SERVER = PONTI

[NIOSRVNV.NIOSERVE.EMS]
SERVER(2)> /iphost
         Char*64     IPHOST_SERVER(2) = ponti.esca.com

[NIOSRVNV.NIOSERVE.EMS]
SERVER(2)> /node
          Char*6       NODE_SERVER(2) = PONTI

[NIOSRVNV.NIOSERVE.EMS]
SERVER(2)>
```

This example shows how to view all of the record contents in one command, “read”, and how to examine one field at a time using the “/” command.

Any field can be examined individually by issuing an “hdbrio” command in the following format:

rio> */field_record(start_subscript:end_subscript)*

If a field from the current record is to be viewed, the record and subscript portions of the command can be omitted. Additionally, if a single instance is to be viewed, the ending subscript can be omitted. To view all of the instances, the wildcard value “*” can be substituted for the subscript values.

The procedure for modifying a field value is similar to reading it with the “/” command. To modify the value, issue the command:

rio> */field_record(start_subscript:end_subscript) = value*

For text fields, the *value* portion of the command may be in quotation marks if the entry is to be case-sensitive or if it contains non-letter characters. Changes can be made to multiple records by using the subscript values.

4.5.3.4 Inserting and Deleting Records

Before inserting a record into a database with HdbRio, you must first position to the correct place in the database. For non-hierarchical records, the new record is inserted immediately following the current record. For hierarchical records, a record is inserted at the correct place in the hierarchy — that is, in order to insert a child record, you must be positioned on the appropriate parent record. For record deletion, the record positioned to is deleted.

The example below shows records being inserted and deleted:

```
hdbrio -a nioserve -f ems niosrvnv
[NIOSRVNV.NIOSERVE.EMS]
rio> list -h
      Record hierarchy of database NIOSRVNV
      =====================================
(n) ITEMS
(f) TEXT
(h) NVDECK
```

```
     |
     +--ALIAS
     |
     +--SITE
         |
         +--CLERK
         |
         +--SERVER
         |
         +--SYSTEM

[NIOSRVNV.NIOSERVE.EMS]
rio> position site = 1

[NIOSRVNV.NIOSERVE.EMS]
SITE(1)> insert clerk

[NIOSRVNV.NIOSERVE.EMS]
CLERK(1)> /iphost="hydra.esca.com"

[NIOSRVNV.NIOSERVE.EMS]
CLERK(1)> /group=60

[NIOSRVNV.NIOSERVE.EMS]
CLERK(1)> /
                    Display of record CLERK(1)
      ==========================================
         Char*64      IPHOST_CLERK = hydra.esca.com
          Char*2       GROUP_CLERK = 60
          Char*6        NODE_CLERK =

[NIOSRVNV.NIOSERVE.EMS]
CLERK(1)> position server=2

[NIOSRVNV.NIOSERVE.EMS]
SERVER(2)> delete
Deleting 1 records (including all subtrees) under SERVER(2)
Delete records  [Y/N]? y
Record(s) SERVER deleted.

[NIOSRVNV.NIOSERVE.EMS]
SERVER(1)>
```

4.5.3.5 Database Maintenance

HdbRio provides two other functions that support database maintenance. These functions are:

- Reset: Recalculates and resets the various pointers contained in the database
- Verify: Verifies that the current database pointers are correct

5. Habitat Management

This chapter provides information about managing your Habitat installation. System management functions are covered, as well as instructions for customizing certain aspects of Habitat and its applications. Topics include:

- Managing a system with the “habitat” command
- Performing a system build
- Managing clones in Habitat
- Customizing Habitat applications, including localization

5.1 Managing a System with the "habitat" Command

Habitat provides a command interface for handling most common management operations. This interface can be used to manage both user-installed files (HABUSER) and system-owned (HABITAT) files.

> **Note:** This command interface is designed to be the same on all operating systems, but it is only partially available in Windows.

5.1.1 The Habitat Command Interface

Habitat provides a straightforward command interface in which you can perform most system maintenance activities. This command interface allows such operations as:

- Starting and stopping Habitat
- Updating the Habitat environment variables
- Building, linking, and loading portions of system-owned (under the HABITAT directory tree) or user-installed (under the HABUSER directory tree) code
- Binding Browser displays
- Managing large numbers of clones

Many Habitat commands can operate on either system-wide (under the HABITAT directory tree) or layered (under the HABUSER directory tree) applications. When a command is intended to operate system-wide, the “-habitat” qualifier must be used. Commands issued without the “-habitat” qualifier do not alter system-owned files; rather, commands issued without the qualifier operate on files in the HABUSER_SRCDIR directory structure, sending their output to HABUSER INCDIR, LIBDIR, or BINDIR.

When used with the “-habitat” qualifier, commands operate on code in the HABITAT_SRCDIR directory structure and send their output to the HABITAT INCDIR, LIBDIR, or BINDIR directories.

Many of these commands can operate on HABITAT or HABUSER codesets. A brief explanation of the source code layout within Habitat is appropriate here. All codesets associated with a particular installation and its layered products should be located in one of two places: the HABITAT_SRCDIR directory structure for codesets owned by Habitat, or the HABUSER_SRCDIR directory structure for layered product and user codesets.

Within these directory structures, the codesets themselves are divided into subsystems. These subsystems are nothing more than logical divisions of codesets; they serve no function other than simplifying management of the codesets. The "habitat" commands shown in the next few sections of this document are often capable of operating either on individual files or on lists of files, at either the subsystem level or the codeset level.

5.1.2 Habitat Command Overview

The "habitat" commands all follow the same general format. Individual commands can support additional qualifiers, which are described in the sections about each command.

habitat *command* -p *phase*

The supported commands and their options are summarized below:

Command	Phase
START	NOSTART, NOSERVER
STOP	
UPDATE	
HELP	Command name
BUILD	PASS1, PASS2, MESSAGE, STATUS, FORMAT
LINK	ALLEXE, SHARED, ALMMSG, SCF
LOAD	DBDEF, DDL, CLS, CASE, APPDEF
CLONE	RESET, REMOVE, REPLACE, CREATE, RETRIEVE, CORE

Some of the commands support qualifiers in addition to the phase qualifier. The next few sections describe each command in detail, including all its associated qualifiers.

5.1.2.1 The Start Command

This command is used to start the Habitat environment. When issued, all the required environment variables are defined, all the shared libraries are available, and the HdbServer (along with any user-specified tasks) is started. The "start" command supports the phase qualifier with two possible values:

- NOSTART: User tasks are not started (i.e., habmgr_start_tasks is not executed).
- NOSERVER: HdbServer is not started, nor are any user tasks.

For a complete description of the startup sequence, refer to chapter 3 Controlling and Configuring Habitat.

5.1.2.2 The Stop Command

When it is issued, the "stop" command shuts down Habitat. The "stop" command does not support any options.

5.1.2.3 The Update Command

On both Windows and Linux, the “update” command is used when a user changes the definition of a Habitat environment variable. This command does not support any options.

On Windows, based on the environment of the command prompt used to run the command, the update command does the following:

- Regenerates the habitat_service_env.cfg file used by the Habitat Windows Service. This file resides in the HABITAT_MANAGER folder.
- Regenerates the habitat_service_env.bat file used for configuring the environment variables for other command prompt windows used for running Habitat command line programs.
- Regenerates the habmgr_define_sys_env.bat, habmgr_define_user_env.bat, and habitat_define_user_env.bat files in the HABITAT_MANAGER folder.
- Updates the HABITAT_VERSION and EMP_VERSION environment variables in the registry for the current user that runs this command.

5.1.2.4 The Help Command

To view help on any supported command, issue the “help” command. For more details, you can specify a particular command as an option. For example, the command:

```
habitat help build
```

generates detailed help about the “build” command. If no option is specified, general help information is given.

5.1.2.5 The Build Command

This command can be used to rebuild all or part of Habitat. It supports a rich set of options and qualifiers. The “build” command is implemented directly on top of standard ESCATOOLS commands, which can be used instead if a user is more comfortable or familiar with them.

Additional qualifiers supported by the “build” command:

- SUBSYSTEM subsystem-name, subsystem-name: Allows you to build particular subsystems only.
- CODESET codeset-name, codeset-name: Allows you to build particular codesets only.
- ORDER wildcard-filename: Builds the codesets in the order specified by the file.
- SKIP skip-filename: Builds the codesets, except those specified in the skip file.
- HABITAT: Indicates that the codesets or subsystems specified should be found in the HABITAT_SRCDIR directories. The results of this build are placed in the Habitat system directory structure (habitat*gg*.habitat...).

These additional qualifiers can be used in conjunction with the standard phase qualifier.

> **Note:** SUBSYSTEM and CODESET qualifiers cannot be used together.

The supported values for the phase qualifier include:

- PASS1: Indicates that PASS1 processing should be done.

- PASS2: Indicates that PASS2 processing should be done.
- MESSAGE: Indicates that MESSAGE file processing should be done.
- STATUS: Indicates that STATUS file processing should be done.
- FORMAT: Indicates that DBDEF formatting should be done.

If no phase qualifier is specified, then a complete BUILD of the specified codesets or subsystems is performed. In Linux, this includes linking of executables after all codesets have been built.

Below are examples of the “build” command and descriptions of the actions taken.

```
habitat build -subsystem rcse -p format,message -habitat
```

This command causes the RCSE subsystem in the HABITAT_SRCDIR directory structure to be processed. Only DBDEF and MESSAGE files are processed.

```
habitat build -codeset netio,alarm -habitat
```

This command causes the NETIO and ALARM codesets, found under the HABITAT_SRCDIR directory structure, to be built.

The results of these commands are placed in the HABITAT INCDIR, LIBDIR, and BINDIR directories.

When using the “build” command, it is important to note that the current set of ESCATOOLS build options is used. Therefore, a user should set the appropriate build options prior to executing the “build” command. For builds of portions of the system, the correct build options can be set by executing the habitat_build_options script located in HABITAT_CFGDIR.

Each codeset included in the command must have a *codeset*.mk file (Linux). Any codeset that does not contain the appropriate descriptor file is not processed.

5.1.2.6 The Link Command (Linux Only)

This command is used to regenerate all or some of the Habitat executables, and it can be used to regenerate shared libraries as well. In Linux, issuing the “link” command may cause the affected codesets to be rebuilt.

Additional qualifiers supported by the “link” command:

- SUBSYSTEM subsystem-name, subsystem-name: Allows you to link particular subsystems only.
- CODESET codeset-name, codeset-name: Allows you to link particular codesets only.
- ORDER order-filename: Links the codesets in the order specified by the file.
- SKIP skip-filename: Links codesets, except those specified in the skip file.
- HABITAT: Indicates that the codesets or subsystems specified should be found in the HABITAT_SRCDIR directories. The results of this build are placed in the HABITAT system directory structure (habitat*gg*.habitat...).

These additional qualifiers can be used in conjunction with the standard phase qualifier.

> **Note:** SUBSYSTEM and CODESET qualifiers cannot be used together.

The supported values for the phase qualifier include:

- ALLEXE: Causes all executables to be linked but does not regenerate the shared libraries.

- SHARED: Causes only the shared libraries to be regenerated. The Habitat qualifier must be used with this phase.
- ALMMSG: Causes only the Alarm message shared library to be regenerated. The Habitat qualifier must be used with this phase.
- SCF: Causes only the SCF shared library to be regenerated. The Habitat qualifier must be used with this phase.

If no options are specified, then the “link” command re-links all of the executables and regenerates the shared libraries (i.e., same as -phase allexe,shared).

Below are examples of the “link” command and descriptions of the results.

```
habitat link -subsystem psa
```

This command relinks all of the applications in the PSA subsystem under the HABUSER_SRCDIR folder, places the resulting executables into the HABUSER_BINDIR folder, and then regenerates all of the shared libraries:

```
habitat link -codeset netio -scm dev5.1 -habitat
```

The command shown above re-links the executables in the NETIO codeset and then regenerates all of the shared libraries. The required link file is fetched from CMS. Before issuing this command, the CMS library containing the codeset NETIO DEV5.1 must be set. The output of the command is directed to HABITAT_BINDIR.

5.1.2.7 The Load Command

When you want to load components into Habitat from the HABITAT_SRCDIR directories, use the “load” command. The “load” command supports several additional qualifiers:

- SUBSYSTEM subsystem-name,subsystem-name: Indicates that elements from the specified subsystems should be loaded.
- CODESET codeset-name,codeset-name: Indicates that elements from the specified codesets should be loaded.
- SKIP wildcard-filename: Allows you to control which codesets are loaded based on an ESCATOOLS skipfile.
- HABITAT: Indicates that the codesets or subsystems specified should be found in the HABITAT_SRCDIR directories.

The supported values for the phase qualifier, when used with the “load” command, include:

- DBDEF: Loads database definitions
- CLS: Loads CLS file definitions
- APPDEF: Loads application definition files
- DDL: Loads FG DDL files
- CASES: Copies savecases to the HABITAT_CASES directory

If no options are specified, then all types of files are loaded.

Below are examples of the “load” command and descriptions of the results.

```
habitat load -p ddl
```

This command loads all of the DDL files contained in all the codesets of the HABUSER_SRCDIR directory structure.

```
habitat load -p appdef,ddl -skip dont_load_these.dat
```

When this command is issued, all APPDEF and DDL files contained in the HABUSER_SRCDIR codesets, except those specified by the file dont_load_these.dat, are loaded.

5.1.2.8 The Clone Command

This command is used to manipulate clones within Habitat. It is always used in conjunction with a clonemgr definition file that controls which clones are created or removed and which cases are retrieved.

The following additional qualifiers are supported by the “clone” command:

- FILE: Specifies the definition file that is used to control the command. This qualifier is not used with the “reset” or “core” commands.
- FAMILY: Specifies which family of clones is manipulated. The family must have corresponding entries in the definition file. If omitted, all families specified in the data file are operated on. This qualifier is not used with the “reset” or “core” commands.
- DIR: Only valid with the “reset” command. It is used to specify the list of directories from which to rebuild the cloning database file. If this qualifier is not specified, only the clones in HABITAT_CLONES are included in the file.

The values supported by the phase qualifier include:

- CORE: This command can be used to re-create the Hdb core file.

  This operation is typically used only to repair a corrupted Habitat system or when a Habitat system has been copied between systems. This phase cannot be combined with any other phase.
- RESET: This command causes the list of clones used by the Hdb subsystem to be reset. It is rebuilt based on a list of directories passed to the command through the dir qualifier. This phase cannot be combined with any other phase.
- REMOVE: Removes the clones as specified by the definition file and the family qualifier.
- CREATE: Creates the clones as specified by the definition file and the family qualifier.
- REPLACE: Replaces the clones as specified by the definition file and the family qualifier. This phase cannot be combined with the “create” or “remove” phases.
- RETREIVE: Retrieves cases in the clones as specified by the definition file and the family qualifier.

Only one of the CREATE, REPLACE, or REMOVE phases can be specified in the same command. The RETRIEVE phase can be used in conjunction with the CREATE or REPLACE phases.

Below are examples of the “clone” command and explanations of the actions taken.

```
habitat clone -p reset -dir dkb200:[esca.clones]
```

This command re-creates the clone directory file used by Hdb to contain the clones found in the directory dkb200:[esca.clones].

```
habitat clone -p create -file emp_esca60.dat -family dts
```

This command creates clones as defined in the emp_esca60.dat file. The file must contain entries for family “dts”.

habitat clone -p retrieve -file emp_esca60.dat

This command retrieves cases into all the clones specified in the emp_esca60.dat file, regardless of what family they are associated with.

5.2 Performing a System Build

The term “system build” means the complete loading, building, linking, and cloning of an entire system. The term typically refers to the processing of only HABUSER codesets.

For more information regarding how to perform system builds on top of a Habitat environment, refer to the *Platform Software Installation and Maintenance Guide* or the *Comm Software Installation and Maintenance Guide*.

5.2.1 Integrating a New or Changed Alarm Message or Status File

While developing your applications, or patching Habitat or its layered products, you might need to build a new or modified alarm message (.almmsg) or status (.status) file into your system. The procedure to do so is covered here because there is a system-wide effect when one of these files is changed.

5.2.1.1 Alarm Message (.almmsg) Files

All .almmsg files in the system are built into a common almmsg object library. The Alarm task requires an index to those modules so that it can reach the correct individual alarm definition for each alarm. Therefore, after any .almmsg file is built into the library, the global index module — almmsg_gblindex — is created from all the alarm message modules in the library and built in, too. Afterward, the modules are all linked into an almmsg shared image/shared library/DLL. The shared library is used only by the Alarm task, but its existence removes the need to re-link Alarm every time an .almmsg file changes.

The details of building and linking in response to a changed .almmsg file differ slightly between operating systems, so we cover each one here.

5.2.1.1.1 Linux Almmsg Processing

Assuming you are set to the directory and codeset containing your new or changed .almmsg file (use *getenv* to get there), you build the .almmsg module and the new global index, and then re-link the shared library, libalmmsg.so, by issuing the command:

```
% build
```

5.2.1.1.2 Windows Almmsg Processing

Assuming you are set to the directory and codeset containing your new or changed .almmsg file (use *getenv* to get there), you build the .almmsg module and the new global index, and then re-link the DLL, habalmmsg.dll, by issuing the command:

```
> build -pass1 xxxx.almmsg
```

5.2.1.2 Status (.status) Files

All .status files in the system are built into a common status object library. Any task that translates a status code into its associated textual information requires an index to those modules so that it can reach the correct translator function. Therefore, after any .status file is built into the library, a global indexing module — scf_getstatus — is created from all the status modules in the library and built in, too. Afterward, the modules are all linked into a status shared image/shared library/DLL. The shared library removes the need to re-link all tasks every time a .status file changes.

The details of building and linking in response to a changed .status file differ slightly between operating systems, so we cover each one here.

5.2.1.2.1 Linux Status Processing

Assuming you are set to the directory and codeset containing your new or changed .status file (use *getenv* to get there), you build the .status module and the new global index, and then re-link the shared library, libscf.so, by issuing the command:

```
% build
```

5.2.1.2.2 Windows Status Processing

Assuming you are set to the directory and codeset containing your new or changed .status file (use *getenv* to get there), you build the .status module and the new global index, and then re-link the DLL, habscf.dll, by issuing the command:

```
> build -pass1 xxxx.status
```

5.3 Managing Clones in Habitat

There are two areas of clone management in Habitat that fall outside the bounds of normal clone creation and removal. These areas are:

- Creating additional clone directories
- Increasing the maximum number of clones

The sections that follow document the procedures required to manage these issues.

5.3.1 Creating Additional Clone Directories

It is frequently necessary to divide the clones in a Habitat system among multiple directories. This is most commonly done for performance reasons, to make the operating system take advantage of multiple disk channels. This section documents the process of creating an additional clone directory, and it provides the necessary information for managing such a scenario.

You can create as many additional clone directories as desired; there is no limit on how many or how few clones can be stored in any particular directory. Hdb uses a special directory file to locate all its clone files. Any clone that is not listed in this file is not accessible to Hdb. This file, clone_directory.cdr, is located in the HABITAT_CDBROOT/cloning_database directory.

The process for creating an additional clone directory is described below:

1. Create a directory with the same file ownership and protection values as the existing HABITAT_CLONES logical directory.
2. With HdbServer shut down, copy the desired clone files to the new directory.

   You do not need to copy clones to this directory. New clones can be added to it from scratch.
3. If clones have been moved, update the clone_directory.cdr file owned by Hdb by issuing the following command:

   `habitat clone -p reset -dir clone-dir1,clone-dir2,...`

   This command causes the file to be re-created based on the clone files located in the specified directories.
4. Restart the HdbServer.

Once the additional directory has been created, new clones can be installed directly to the new directory by issuing the correct “hdbcloner” command:

`hdbcloner -c create_clone -a appname -f famname -l dirname`

Hdb makes the correct entry in the clone_directory.cdr file automatically. Removing or replacing clones created in alternate directories does not require any additional qualifier.

> **Note:** Clones can only be created in new directories. They cannot be moved by issuing the “replace” command.

5.3.2 Increasing the Maximum Number of Clones

A parameter passed to HdbServer when it is started controls the maximum number of clones that can be in use with Hdb. By default, this value is 150. To increase the number of clones supported on the system beyond 150, the HdbServer must be restarted using the -mxclones qualifier to specify the new maximum number of clones:

hdbserver -mxclones 250

For instructions on making these changes permanently, refer to the chapter in this document specific to your operating system.

5.4 Customizing Habitat Applications

In several areas, users can customize Habitat applications after a delivery. These same customizations are possible, through the same methods, for any application delivered by Habitat or Platform.

The topics addressed in this portion of the document include:

- Customizing database sizes
- Modifying application quotas and privileges
- Localizing messages and status values

> **Note:** This document does not describe all of the details associated with customizing applications. For a complete reference, refer to the *Habitat Programmer Guide*.

5.4.1 Customizing Database Sizes

Unlike previous versions of Habitat, changing the dimensions of a database without recompiling the code is not possible (except for the PERMITNV database of the PERMIT application; for details and limitations, see the *PERMIT User Guide*). Since the source code is not delivered with Habitat applications (except for Alarm), the database sizes of Habitat applications cannot be changed. If you require a change to the dimensions of a database owned by Habitat, contact GE Customer Support. For information about customizing the Alarm application, refer to the *Alarm User Guide*.

5.4.2 Modifying Application Quotas and Privileges

The quota and privilege values for each task in Habitat are stored in the application definition (appdef) file. Application quotas and privileges are set either by a specific APPDEF file contained in the application's codeset or, when no APPDEF file has been created for the application, by the default application definition file stored in the APPMGR codeset.

> **Note:** Carefully consider the impact on system resources before modifying an APPDEF file. This is especially true for modifying the default APPDEF file, since many tasks in Habitat acquire their privileges and quotas from this file. If an application that uses the default APPDEF file requires a substantial increase in quotas or privileges, it is often better to modify the specific APPDEF file for the application rather than modifying the default APPDEF file.

After modifying the APPDEF file, you must load the definition into Habitat. This can be accomplished with the command:

```
habitat load -p appdef -codeset codesetname
```

where *codesetname* is the name of the codeset containing the new or modified application definition file.

Note that, if the codeset in question resides in a subdirectory of HABITAT_SRCDIR, the "-habitat" qualifier must be used with the command.

Any changes to application definitions should be stored in a source code management system.

An example of a typical APPDEF file is shown below:

```
TASK mlfdmn
   DESCRIPTION "Message Logging and Formatting Daemon"
   TARGET DEC_LINUX
      LOCATION "HABITAT_BINDIR"
      PRIORITY 23 # to be above the normal (19 is the default)
   END_TARGET
   TARGET WIN_NT
      LOCATION "HABITAT_BINDIR"
   END_TARGET
END_TASK
```

The APPDEF file allows you to control individual quotas, privileges, and priorities on an operating system basis.

5.4.3 Localizing Messages and Status Values

Nearly all Habitat and layered product applications (Platform) log messages into Habitat databases via the MLF API. This API uses message definition files (*.message) to determine which text is generated in the Habitat logging database when the application code issues a particular message.

One benefit of separating the actual text of the message from the coding of the application is that it permits a user to localize the messages without having to recompile the entire application. For a complete description of the localization process, refer to the *Message Logging Facility User Guide*. The same is true for the text associated with status codes. This text is contained in the status definition file (*.status) and it can be localized in the same way the message files can be.

6. User Interface Management

The *Browser Software Installation and Maintenance Guide* and the *Browser User Guide* provide detailed information about Browser.

7. PROCMAN and the "HABITAT" PROCMAN Model

This chapter describes and explains the PROCMAN model delivered with Habitat, as well as the relationship of the Habitat tasks with PROCMAN.

7.1 The Role of PROCMAN

PROCMAN is a subsystem within Habitat that is responsible for managing a collection of tasks. In this context, the term “managing” means deciding when a task should be started, the order in which tasks should be started, if and when a task should be stopped, and the sequencing and scheduling of work for those tasks that are not continuous in nature. All of this information is modeled in a database, so that it can be extended and changed without requiring code changes in the applications that are being managed.

PROCMAN provides support for breaking down the work that a program does into “nodes”. The simplest analogy is to think of a node as a subroutine within the program. You specify the order in which the subroutines are to be called from outside the program — in the PROCMAN model. Taken to its extreme, such an approach increases the complexity of a system rather than reducing it. So, instead of thinking about a node as a single subroutine, you should think about it as a well-defined amount of work. Typically, you break a task into more than one node because you want the flexibility to cause other work within other tasks to commence when the processing within a node completes.

Based on the model in the PROCMAN database and the dynamic behavior of the system as a whole, PROCMAN can make decisions as to which nodes should be executed when and in what order. It also supports schedules so that you can cause a particular node to be executed at a periodic interval — say, every two seconds, or once per day at midnight.

7.2 The "HABITAT" PROCMAN Model

The standard PROCMAN model delivered with Habitat provides a minimal set of tasks required to get the user interface working, to form NETIO paths, and to enable message logging and formatting. This model is intended to provide a minimal environment for running the user interface and performing development activities.

The “HABITAT” PROCMAN model controls the following tasks:

- The Hdb server (HDBSERVER) task: By default, the Hdb server is only started by PROCMAN in Windows. It is required in order to access any clone of the system.
- The HDB server monitor (HDBMONITOR) task: It ensures proper completion of the Hdb server initialization sequence before unleashing the other tasks, and it monitors the HdbServer.
- The PROCMAN clerk (PROCDBCLERK) task: The clerk is not required, but it updates the PROCMAN database used by the PROCMAN displays.
- The Browser Data Server (WFGSERVER) task
- The PERMIT Server (PERMSRV) task
- The Ruser Server (RUSERSERVER) task
- The NETIO Address Registry Clerk (NIOARC) task
- The NETIO Clerk (NIOCLERK) task

- The NETIO Server (NIOSERVE) task
- The Message Logging and Formatting Daemon (MLFDMN) task
- The Data Entry (RFGDICE) task

The “HABITAT” PROCMAN model is usually started through a “tview” command or by clicking on the Start Habitat icon in Windows.

Note that the “HABITAT” PROCMAN model is run in the HABITAT family and, therefore, the following environment variable is defined as part of the standard installation:

procman_model_habitat

For more information about the environment variable PROCMAN_MODEL_*family*, refer to the *PROCMAN User Guide*.

7.3 Customizing the PROCMAN Model

The delivered model can be customized to fit the needs of a development environment or any other environment. Any PROCMAN customization should be performed carefully and according to the guidelines provided in the *PROCMAN User Guide*.

7.4 Habitat Tasks and Their Relationships with CFGMAN and PROCMAN

The following table identifies the Habitat–supplied tasks that you might consider modeling in PROCMAN or Configuration Manager, or might have up and running whenever Habitat is running. The purpose of the table is to communicate in a single place the high-level attributes of these tasks, so you can successfully control them using PROCMAN and Configuration Manager.

Subsystem – Task	PROCMAN	Cfgman Interface	Notes
Hdb – Hdbserver	Passive	No	Most systems start hdbserver outside of PROCMAN as part of Habitat startup and rely on hdbmonitor to detect its failure. However, it can be started by PROCMAN if you wish.
Hdb – Hdbmonitor	Active	No	Communicates with Hdbserver and aborts if it detects that hdbserver has exited. Intended to be started by PROCMAN as a surrogate for hdbserver, which is assumed to be started before PROCMAN.
Hdb – Mrs	Passive	Yes	
Hdb – Casesrv	Passive	No	
PROCMAN – PROCMAN	N/A	Special	PROCMAN and CFGCTRL monitor each other.

Subsystem – Task	PROCMAN	Cfgman Interface	Notes
PROCMAN – Procdbclk	Active	No	The PROCDBCLERK task must be modeled in PROCMAN.
Logman – Logman	Passive	Optional	The CFGMAN interface is enabled/ disabled by a global item in the LOGMAN database visible from the Logman Definition display. If the interface is disabled, then Logman assumes the functions of the enabled role. Logman reports the status of printers to CFGCTRL as EQUIPs.
PLCMGR – Plcmgr	Passive	Optional	The CFGMAN interface is enabled/ disabled by a global item in the PLCMGR database visible from the PLCMGR definition display. If the interface is disabled, then PLCMGR assumes the functions of the enabled role.
Habconnect – Sampler	Passive	Optional	The CFGMAN interface is controlled by a global item in the SAMPLER database called “NOPROCMN”. The field is not currently shown on a display. If the interface is disabled, then SAMPLER assumes the functions of the enabled role.
Rfgdice	Passive	No	
Alarm – Alarm	Active	Yes	Alarm can be run without PROCMAN or Cfgman.
Cfgman – Cfgctrl	Active		
Cfgman – Cfgmoni	Active	Yes	Accepts a role.
Cfgman – Cfgdisk	Passive		
Cfgman – Cfgtstat	Passive	Yes	This task reports the status of the TrueTime device as an EQUIP to CFGMAN.
Cfgman – Cfgping	Active	Special	This task reports the status of the local host’s network interface(s) as DEVICEs and the status of other network objects as EQUIPs.
Cfgman – Cfgguard	Not started by PROCMAN	Special	This is a special task that monitors CFGCTRL to detect if it stalls or hangs. If this happens, CFGGUARD crashes the machine. This task is started by CFGCTRL itself — not by PROCMAN.
Cfgman – Cfgjmx	Passive	Yes	
Cfgman – Cfgsite	Passive	Yes	
Ui – Wfgserver	Passive	Optional	Supplies data to Browser clients.

Subsystem – Task	PROCMAN	Cfgman Interface	Notes
Ui – RuserServer	Passive	Yes	Handles the passing of “ruser” commands from Browser clients to the correct application.
PERMIT – Permsrv	Passive	Yes	Controls the user authorization and provides mode management. Note that Permsrv must not be run on a Standby machine when Cfgctrl is not running.
Mlfdmn – mlfdmn	Passive	No	

8. Redundant Instances of Habitat

This chapter introduces the concept of a dual-redundant system. The applications within Habitat that help a user manage redundancy are described. Also included in this chapter is a discussion of data recovery and replication services.

8.1 Dual-Redundant Systems

In a typical Habitat–based Energy Management System (EMS), every vital component of the system has a backup. From EMS hosts and networks, to printers and plotters, there are two of everything. The rule of thumb is for there to be no single point of failure.

A simple configuration might look like the following:

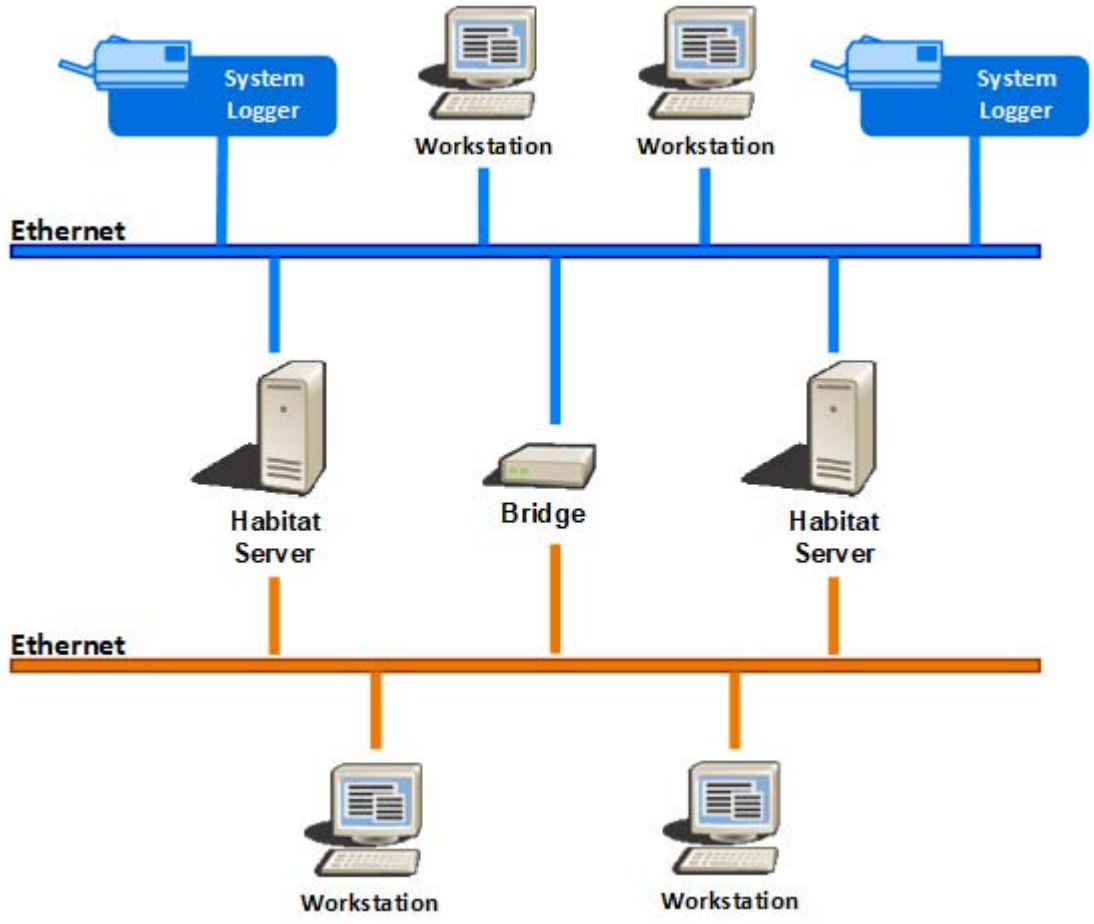

Figure 10. Dual-Redundant Configuration

In a full EMS system, there are other components, such as Telemetry Front Ends (the Control application); these components are duplicated as well. On most systems, the UI workstations also have dual network connections.

Habitat provides capabilities directed at supporting dual-redundant systems. Applications recognize that they may be running on the active host (the Primary) or on the non-active host (the Standby). A

special application, Configuration Manager, is delivered as part of Habitat to manage which role (Primary or Standby) each application should take on.

Complementing the Configuration Manager (CFGMAN) is the Memory Replication Services (MRS) application. The MRS application handles synchronization of the data in the various applications between the Primary and Standby systems. CFGMAN monitors both the hardware and the software running on each server, determines whether the hardware and/or software is available to perform its work, and assigns it an appropriate role. If CFGMAN detects that the Primary system is no longer able to function properly — for example, because a piece of hardware failed or a critical application aborted — it causes a failover. A “failover” is a switching of the Primary and Standby roles.

The remaining sections of this chapter provide an overview of how a dual-redundant system operates and what you must do to manage a redundant system. Habitat applications that play a role in managing a dual-redundant system are introduced and explained.

8.2 Data Recovery and Replication

For two systems to work together — one as the Primary and the other as the Standby — the databases for critical applications on each must contain identical data, and the coherency of this data must be maintained. That is, as data is modified on the Primary, the databases on the Standby must be kept synchronized. This is complicated by the fact that some applications (Alarm) require that data not only be current, but be historically correct.

The Habitat application designed to fulfill these requirements is called “Memory Replication Service” (MRS). The initial synchronization of the databases is called “recovery”, and the process of keeping them synchronized is called “replication”.

8.2.1 Startup and Recovery

The following discussion assumes that there are two EMS host machines, that data replication services have been correctly configured, and that the systems are under CFGMAN control.

The first system to complete its startup procedure takes on the Primary role. Its tasks are assigned the Primary role by CFGMAN, and the system begins processing. Once the secondary system has been started, the CFGMAN task on the secondary forms a connection to the one on the Primary. The CFGMAN on the secondary machine is told that one machine already has the Primary role and that the new machine should take on the Standby role. After all tasks on the secondary machine start, CFGMAN instructs MRS to begin recovering the databases from the Primary to the new Standby.

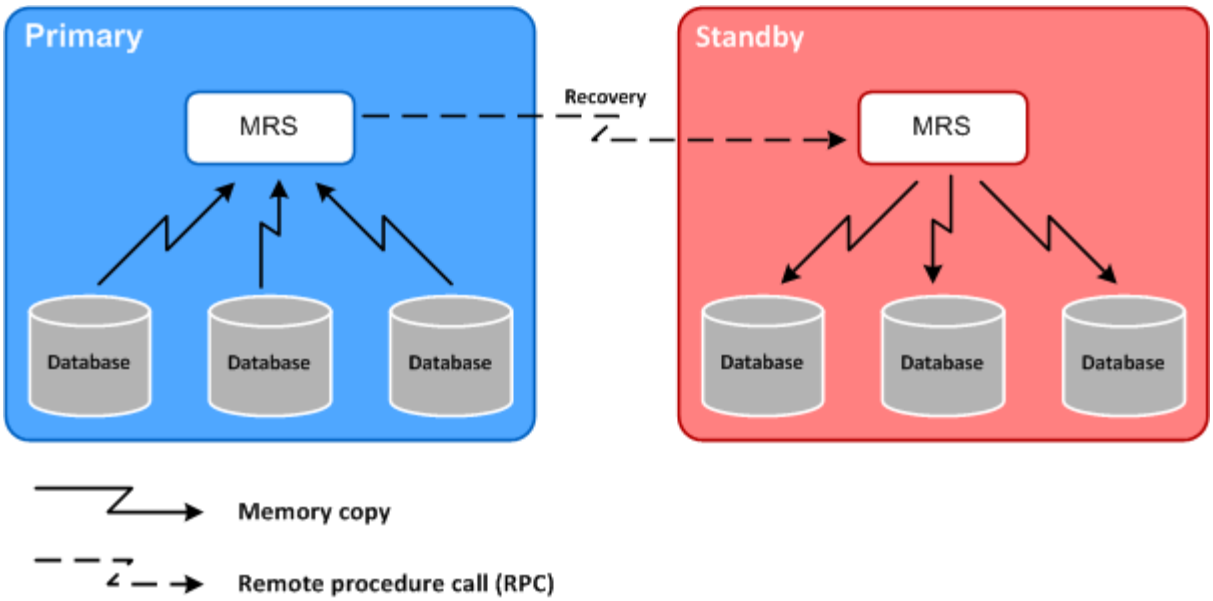

Figure 11. Startup

Each machine has an MRS process. If the machines switch roles (have a failover), the active and inactive MRS processes on each machine also switch roles.

> **Note:** Within the CFGMAN application, the system administrator configures which databases require recovery and replication. Only the databases so marked are recovered. Only when recovery is complete is the Standby machine truly ready to take on the Primary role in the event of a failover.

8.2.2 Replication and Failover

Once both machines are up and running in their assigned roles, they must continue to keep their databases synchronized. As data arrives at the Primary from the RTUs and as calculations are performed by the power applications, this updated data must be kept current on the Standby system. MRS permits this to happen without direct action by the application programmers.

By designating partitions within its database as “replicated” and by installing the clones with a special “replicated” flag, Habitat’s database system (Hdb) automatically replicates the appropriately marked data on the Standby machine. This is possible because calls to the MRS API are embedded in Hdb. As a user writes data to the database (through a transaction), Hdb automatically determines if the data should be replicated. The diagram below illustrates this process:

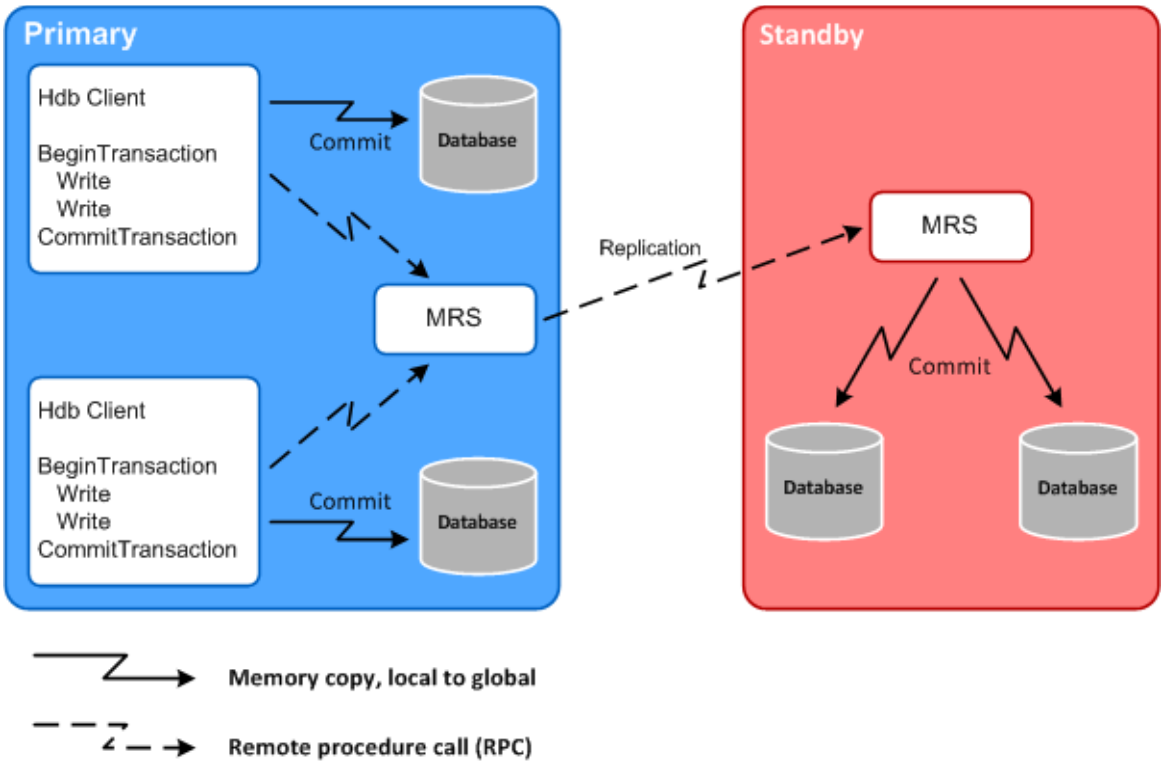

Figure 12. Data Replication

8.3 Managing Redundant Systems in Habitat

A number of applications are involved in managing a redundant system. They include Configuration Manager, MRS, PROCMAN, and Alarm.

The following sections briefly describe the role of each of these applications.

8.3.1 Configuration Manager

The task of Configuration Manager is to monitor the hardware and software that make up the system, as well as assigning roles (Primary, Standby) and managing database recovery. The Configuration Manager application has a number of tasks to support these operations:

- CFGCTRL: Called “Config Control”, this is the main process in CFGMAN. The status of all hardware and software components in the system is reported to CFGCTRL. CFGCTRL determines if the system is available to take on a role and, by communicating with the CFGCTRL on the other server, it determines which role each system should take on. CFGCTRL also tells MRS when to perform database recovery. MRS registers each clone that it finds as replicated with CFGCTRL.
- CFGMONI: Called “Config Moni”, this task monitors the status of the system. Each CFGCTRL process sends information to each CFGMONI process regarding the status of the components in the system.

  Note that CFGMONI is used only as a user interface. CFGCTRL is responsible for making the availability and role assignment decisions. CFGMONI is provided so that you can look in a single place to see the status of all systems (including, for example, Platform front ends).

- CFGDISK: This process monitors the availability of the disks on each host system. The system integrator chooses which disks are monitored and whether the system should fail if any of the disks fail. CFGDISK reports the disk status information to CFGCTRL.
- CFGPING: This process monitors the availability of network connections among the EMS hosts, TFEs, and (optionally) the UI workstations. It reports the network status to CFGCTRL.
- CFGTSTAT: This process monitors the time source devices. It reports their status to CFGCTRL.

These are the main components of CFGMAN. Together, they can monitor the health of the system and determine which EMS host should take on the Primary role. The *Configuration Manager User Guide* contains a great deal of information about redundant systems, including how to model which databases should be recovered.

8.3.2 MRS

The MRS application is responsible for database recovery and replication. It is made up of two processes and an API. The MRS API is used only by Hdb and Alarm; application programmers need not make direct use of this API.

When instructed by CFGCTRL, the MRS on the enabled system initiates recovery (a complete update) of the appropriate database. MRS receives notification from Hdb whenever an application writes data in a partition that has been marked “replicated”. MRS sends this replicated data to the MRS process on the Standby EMS host. In the event of a failover, MRS on the old Primary transitions into the standby role and the MRS on the new Primary becomes enabled.

8.3.3 PROCMAN

PROCMAN does not normally play a large role in managing redundancy. It is informed by CFGCTRL as to whether it is running on the Primary system. PROCMAN takes no specific action when informed that it is running on the secondary (Standby) system. If CFGCTRL itself fails, PROCMAN forces the system to reboot. PROCMAN normally ensures that all tasks remain available by restarting them in the event of failure. For more information about the role of PROCMAN in redundant systems, refer to the *PROCMAN User Guide*.

8.3.4 Alarm

Alarm plays a special role in a dual-redundant system. Alarm contains critical time-series information that must be maintained exactly, without duplication or omission, on both the Primary and Standby systems. Additionally, events can be generated by clients running on the Standby system. These events must also be integrated into the single time-ordered list of events that is maintained by Alarm.

Even more critical is the fact that Alarm events are tied to specific values of Platform data. That is, when data changes in a manner that causes Platform to issue an Alarm, the Alarm event and the corresponding data must be updated consistently on the Standby. Either both must arrive or both must not.

The diagram below shows how Alarm clients, the Alarm server, and MRS interact:

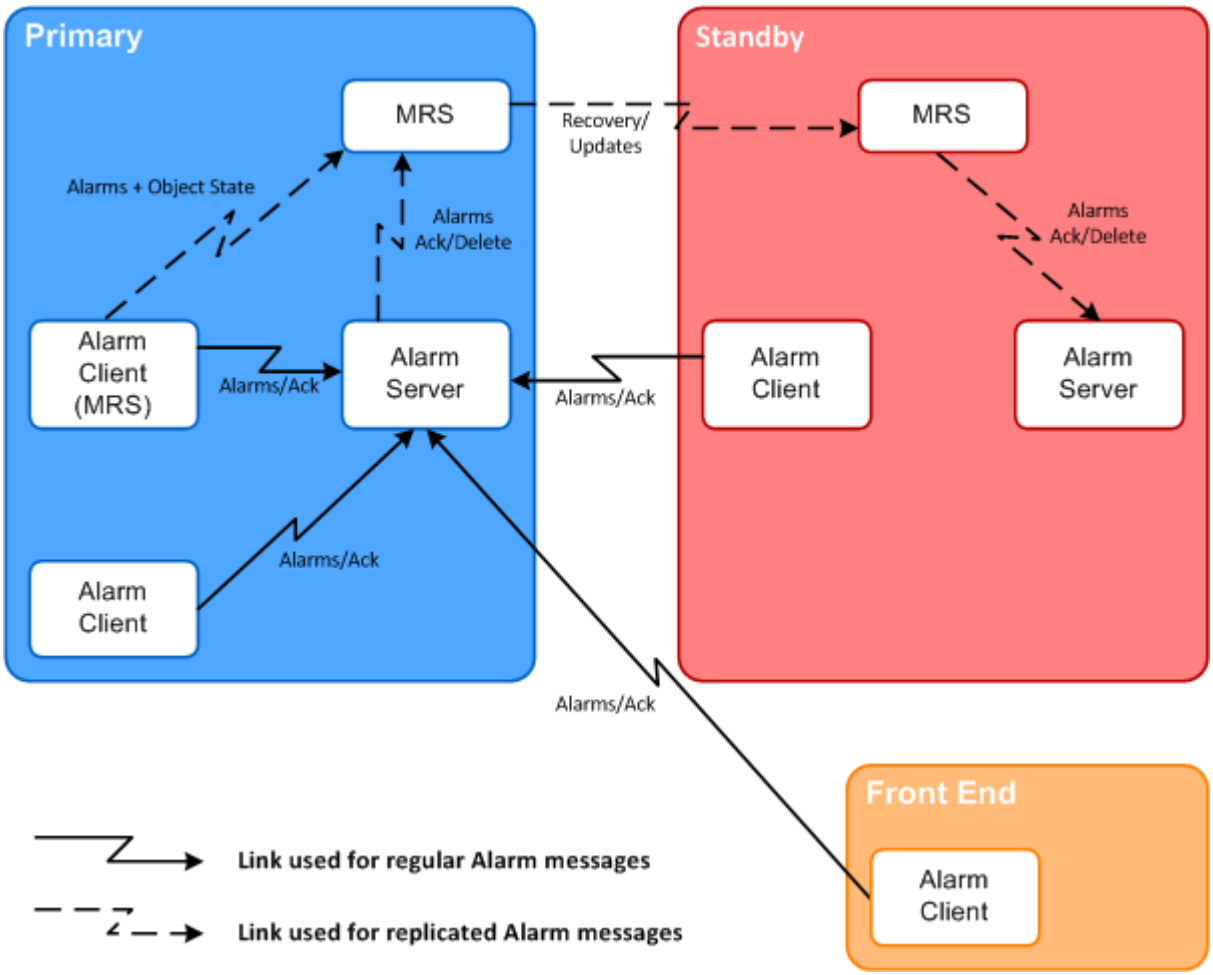

Figure 13. Alarm Interaction

For more details about how Alarm interfaces with MRS, and how events are replicated between the two systems, refer to the *Alarm User Guide*.

9. Other Habitat Applications

There are a number of applications included in Habitat whose roles have not yet been described or described completely in this *User Guide*. Many of these applications are included in Habitat to support other layered products, such as Platform. This chapter gives brief descriptions of these applications.

9.1 Configuration Manager

Within a real-time control system, Configuration Manager and its components provide many of the control and configuration features required to support a dual-redundant system. These components include:

- CFGCTRL: Monitors and controls the availability and role assignments for Platform and Habitat components.
- CFGMONI: Provides a user interface for viewing CFGCTRL data for both the primary and secondary hosts.
- CFGPING: Performs network availability monitoring.
- CFGTSTAT: Provides time synchronization services to the system.
- CFGDISK: Provides disk monitoring information.

For more information about Configuration Manager, refer to the *Configuration Manager User Guide*.

9.2 Alarm

The Alarm application provides a method of notifying operators, in real time, of events and alarms occurring on their system. “Events” are changes to the system’s state that are detected by application software.

An application that detects an event can use Alarm to bring the event to the operator’s attention. An event is usually printed and archived by the Alarm application and it may be considered an alarm. An audible and/or visual indicator is used to report the alarm to the operator. Warning tones associated with Alarms are generated by Browser.

An alarm requires operator acknowledgment and/or deletion.

Each event is typically associated with a database object (e.g., a Platform point representing a circuit breaker, a PROCMAN task). Each event is also posted with a set of attributes; this set is used by the Alarm application to process the event according to these attributes and the modeling information stored in a database.

For more information, refer to the *Alarm User Guide*.

9.3 NETIO

The NETIO application provides communications services to Habitat and Platform. With NETIO, tasks can form intra- and inter-node connections without detailed knowledge of the network configuration. The components of NETIO and their functions include:

- NIOARC: The NETIO Address Registry Clerk provides a mapping between TCP/IP address — port number pairs and network access point names in text strings.

- NIOCLERK: The NETIO Clerk is the connection point between NETIO clients (applications) and the rest of the NETIO network. It acts as an intermediary between the clients and the NETIO server. Clients can connect to NETIO Clerks on their own or to remote hosts via an environment variable.
- NIOSERVE: This task performs the actual work of matching client requests and sending the data required to form actual network connections out, via the NETIO Clerk, to the NETIO clients around the network.
- NETIO API: This programming interface is used by the clients to simplify the process of establishing network connections. Through this API, the client makes simple requests — declaring what network object name they wish to be known by, and what other network objects they wish to connect to — that result in automatic communications between the client and the NETIO Clerk. This API also provides simplified communications functions. It is asynchronous in nature, allowing clients to make requests and to be informed of events without polling.

9.4 Logman

Providing access to system printers, LOGMAN provides both a task that monitors and controls the printer queues and an API that permits clients to make simple printing requests.

Various software components of a control system generate many useful events for system monitoring. These events are printed for user notification and for historical purposes. In order to ensure that all events print, it is necessary to automatically provide backup printer capability. In the event of a printer failure, Logman satisfies this need for highly available printing of events.

9.5 Sampler

Sampler is a process that runs on the host Habitat system to provide data from Hdb databases as requested by HABDDE. When Sampler receives a request, it accesses the appropriate database, obtains the data, and sends it to HABDDE.

One Sampler process can provide support for multiple personal computers simultaneously, which can include personal computers communicating by TCP/IP. Sampler can be started and controlled by PROCMAN in a dual-redundant configuration.

Sampler can access any Hdb database on the system. It does not use Habitat–included files, and therefore it is not tied to a particular version of a database. Support is provided for two- and three-dimensional Hdb database structures, indirect pointers, Habitat–linked list structures, and message logs (circular lists).

For reading from Hdb databases, Sampler can optionally use the Habitat PERMIT database to control personal computer access. For writing to Hdb databases, use of the PERMIT database is required.

9.6 PLCMGR

PLC Manager (PLCMGR) is a Habitat PPM-compliant application that enables communication between other Habitat applications and a Programmable Logic Controller (PLC). PLC Manager provides a standard interface (the PLCMGR API) through which client applications can send commands to a PLC without using the low-level protocol required by the PLC.

PLC Manager notifies the client application of the outcome (success or failure) of its request.

The PLC Manager (PLCMGR) utility drives output devices that are controlled by analog or digital signals. PLCMGR supplies these signals via standard output modules in Programmable Logic Controllers (PLCs).

PLCMGR is designed to support the Siemens SIMATIC TI545 Programmable Logic Controller, which uses Series 505 Modules for I/O interfaces.

Devices that can be driven by PLCMGR include strip chart recorder pens, alarm horns, and parallel digital switches.

The main advantage of PLC Manager is that low-level communication with the PLC is hidden from the application. The application developer does not need to know the intricacies of the PLC's communication protocol in order to control PLC devices. In addition, if the PLC model or protocol changes, only PLC Manager needs to be modified in order to support the new form of communication.

PLC Manager also allows multiple applications to communicate with the PLC without the applications interfering with one another.

9.7 Casesrv

The casesrv application exists to support savecase operations from the user interface. The casesrv application has a database that stores information about the savecases that reside in HABITAT_CASES.

9.8 Timedate

The Timedate utility exists to answer needs prevalent in any real-time system portably and efficiently. Real-time systems need to manipulate and query time. They need to schedule future events, review and display past events, gather time-sensitive data (such as accounting and resource utilization data), and compare and order events by time.

Operators of such systems need to see time displayed in a manner that reflects the real world (like a wristwatch) but which is coordinated beyond time zone borders. Integrators need to be able to provide operators with time information in readily understandable forms, appropriate both to the particular situation and to the local language and customs.

Engineers using a real-time system sometimes need to suspend "normal time" in order to re-examine past event scenarios, to investigate future contingencies, or to do training. Such operations may require their own clocks and complete control over the times returned by such clocks, without affecting the time perceptions of other processes on the system.

Timedate addresses all the above issues via an implementation common to both Linux and Microsoft Windows.

9.9 Trend

Trend allows you to collect and maintain data from real-time or study applications, either within the Energy Management Platform (Platform) or from other Hdb database–based applications.

The chart control component inside Browser allows you to present the data in graphical form in order to spot and study trends in your system's performance.

10. Software Configuration Management

This chapter provides high-level guidelines regarding how project engineers should manage the software they receive from GE as well as the applications they can develop on top of Habitat.

Software Configuration Management (SCM) involves keeping track of software changes so that anyone, at any time, can re-create the state of a software system. Not only can anyone re-create the current system from source code, but proper use of the SCM tool allows them to re-create that state at specific points in the past as well. Further, SCM tools make it easy to identify what you have changed since a particular time. Depending on the tool, you may be able to "merge" changes into future releases — thus eliminating the need to re-implement changes when a new release comes out.

Habitat does not include an SCM tool. There are several SCM tools available in the marketplace; your choice depends on the operating system you are using, the cost, the availability of the tools, and your needs. This chapter assumes that you have an SCM tool, and it makes general recommendations about the tool's use relative to tracking Habitat product files.

The consistent use of an SCM tool to track changes is central to successful configuration management. All programming staff who work with the software should be familiar with the SCM tool selected by the project team.

10.1 Organization of Code in a Habitat Environment

As described in earlier chapters, Habitat is laid out across several physical directories according to a precise organization. This organization is primarily driven by a desire for both a physical and a logical separation of the files owned by Habitat itself and the files owned by layered software. The HABUSER space of the directory structure is provided to receive the layered applications, while the HABITAT space should remain untouched as much as possible.

Projects should not deviate from this organization. Choosing a different organization is technically possible, but all Habitat command procedures assume that such an organization is being used. All subsequent releases also assume that the directory structure is standard.

10.2 Customizing a Habitat Release

The purpose of this section is to identify the ways in which a customer should go about modifying a Habitat environment, to reduce the effort involved with upgrading to future versions of Habitat.

Even though the Habitat source code is not delivered, there are many ways to customize the Habitat environment. For example, displays can be modified, quotas in the application definition files can be tuned, and savecases may require updating.

The basic philosophy regarding customization of Habitat is to avoid modifying the "HABITAT" directories and, instead, to place all customizations in the "HABUSER" directories. There are several benefits to this approach:

- Habitat patches and upgrades can be safely applied with no need to re-apply a customization to a file located under the HABITAT space.
- Layered products, such as Platform, very likely include tools to build and manage the code under HABUSER and thus they can pick up customizations located there.

- Typically, all the content of the HABUSER space is stored and tracked using the project's software configuration management tool[1]. The same could be done for the HABITAT space, but this is less frequent and is not required if no files are modified.
- Propagation of the HABUSER directories from machine to machine, such as from a development machine to a production machine, is easier to perform if nothing must be done for the HABITAT space.

The following sections discuss typical ways in which Habitat can be customized.

10.2.1 Customizing Savecases

Savecases delivered with Habitat should not be modified. If you need a savecase that is very similar to a default savecase, a new savecase with a different title should be created and stored in a codeset located under the ...\habuser\special subsystem.

10.2.2 Customizing Database Definition (DBDEF) Files

In general, it is not possible to modify the Habitat database definitions, with the following exceptions:

- The ALARM and ALARMLST databases: The only source code delivered with Habitat is for the Alarm application. See the *Alarm User Guide* regarding how to resize or customize the Alarm databases.
- The PERMITNV database: This database is used to model user and mode definitions. It is used by the PERMIT server, which has been designed to cope with database sizing changes for records that are not impacting the PERMIT database (thus area, application, and console sizing cannot be changed).
- The TRENDBUF database: It is used by the Trend server, which has been designed to cope with database sizing changes without requiring a recompilation.
- The message log databases can be resized.

Except for the Alarm databases, the only possible changes regarding the databases mentioned above are related to record sizes. Alarm databases can be changed because the source code is provided.

The ability to specify sizes outside of the DBDEF files is provided via MXDEF files and is the preferred method to resize a Habitat database. MXDEF files created to modify a Habitat database should be stored in the directory identified by the HABITAT_MXDEFDIR environment variable. For more information regarding the MXDEF files, see the *Hdb User Guide*.

10.2.3 Customizing Application Definition (APPDEF) Files

Depending on your operating system and how you are using Habitat, you may need to modify the priority and/or quota information associated with a Habitat task.

However, you may need to modify task attributes for tasks that do not run under PROCMAN control or that are only defined in the APPDEF files. The tuning parameters that are shipped with Habitat are

[1] It cannot be stressed enough that all changes should be stored and tracked using the project's software configuration management tool. In those file types that support comments, each change should be noted using a revision history note in the header of the file.

based on minimum values required to install and operate GE's Platform product using the test models.

The recommended approach to customizing an APPDEF file is to make a copy of the file under a codeset located in the habuser\special subsystem and modify the file there. If a "loadhab" is performed on the HABITAT directories after the HABUSER directories, then the HABITAT-delivered APPDEFs are built instead of the customized ones. To prevent this, it is recommended that you rename the HABITAT APPDEF files that have the same name under HABUSER into *.appdef_save so that they do not get "picked up" by the "loadhab" command. However, patches must be studied carefully, since they can contain APPDEF files.

10.2.4 Customizing User Interface (DDL) Files

Displays and their respective pictures are stored in text files (DDL files). Many Habitat applications contain displays that you may wish to modify by editing the DDL files (using the Display Builder).

There are two classes of customization:

- Creating a new display: New element and display set files should be created and stored under the habuser\special subsystem. Navigation to the new display should be provided via the menu resource file. If capabilities for navigating to the new displays must be provided within some Habitat displays, some changes to one or more Habitat element sets may be required. In this case, the Habitat element sets to be modified should be duplicated under the codeset containing the new display.
- Modifying an existing display: The entire DDL file impacted by these changes should be duplicated under a codeset in the habuser\special subtree, and the Habitat files should not be altered.

The above approach introduces an undesirable build dependency. If a "loadhab" is performed on the Habitat directories after the HABUSER directories, then the Habitat–delivered displays are built instead of the customized ones. It is better to rename the DDL files delivered with Habitat with an extension such as *.ddl_save so that they are not recognized by the "loadhab" command. Then the customization is applied under the HABUSER directories. However, patches must be studied carefully, since they can contain DDL files.

> **Note:** If the Alarm displays must be customized, it is recommended that you copy the entire Alarm codeset under HABUSER, as described in the *Alarm User Guide*.

10.2.5 Adding New Applications to the System

Adding applications to a Habitat environment is straightforward. The HABUSER space is provided for this purpose.

> **Note:** It is highly recommended that the application software be ESCATOOLS-compatible, in order to take full advantage of the Habitat environment.

The Habitat command is designed to handle all ESCATOOLS-compliant applications placed in the HABUSER space. For more information about ESCATOOLS, see the *ESCATOOLS User Guide*.

10.2.6 Customizing Alarm

The Alarm subsystem is unique within Habitat, because both the binary and the source code are shipped with the product.

By default, when you install Habitat, you get the binary version of Alarm — just as you get the binary code for PROCMAN, Configuration Manager, and all the other subsystems within Habitat.

The reason why Alarm source code is included in the release is that some projects require modifications to Alarm. Most of the time, the changes are limited to dimension changes — but sometimes there is a need to modify or extend Alarm.

Since Alarm is part of Habitat, care must be taken as to how it is modified, for two reasons. First, it is imperative that your modifications not be confused with the delivered product. Second, it is imperative that you do not break any of the interfaces that other Habitat subsystems rely on.

The following guidelines, if followed, increase your success and reduce long-term costs associated with your customizations of Alarm:

- Make sure to check with GE about the possibility of including the required customization in the product, and/or have the customization implemented by the Habitat staff. Alarm is a complex application, very sensitive to performance, and thus customizations are often difficult and may have unforeseen side effects.
- Only change what you must. More than most subsystems, Alarm changes from release to release. If you want to pick up the new features, your customizations must be merged or re-implemented in future releases. If you minimize your customizations, you will minimize your future upgrade costs.
- Remember that customizations make it more difficult for Alarm experts to help you resolve problems with Alarm.
- The recommended approach is to create an ALARM codeset under the ...\habuser\special subsystem and manage the customization there. The *Alarm User Guide* provides additional details regarding how to customize Alarm, and it should be consulted.
- Always think of the future cost of maintaining your customizations. Be as modular as you can be; it reduces the work involved with merging your changes into future Habitat releases.

11. Frequently Asked Questions

This chapter contains answers to frequently asked questions about Habitat, along with how-tos, descriptions of certain tools, and help with some operational problems.

11.1 How To

Many of the commonly performed Habitat operations are summarized below.

- Start Habitat – Starting Habitat is a simple operation:

  ```
  habitat start
  ```

- Stop Habitat – Stopping Habitat is similar to starting it:

  ```
  habitat stop
  ```

  > **Note:** Using a "tview stop/all" command to shut down Habitat is **not** the recommended procedure. Using the "habitat stop" command does several things that the "tview" command does not. Using the "habitat stop" command shuts down PROCMAN in a controlled way. It also cleans up temporary files that may create problems when restarting Habitat.

- View what is running – Use the TVIEW utility to view running tasks:

  ```
  tview list
  ```

- Change the default Habitat context:

  ```
  context application family
  ```

- Create a clone:

  `hdbcloner -c create_clone -a appname -f famname`

  where *appname* is the name of the application and *famname* is the desired family name.

- Retrieve a case:

  ```
  hdbcopydata -sf archivename -d appname.famname
  ```

  where *archivename* is the complete name (including the path) of the case, and *appname* and *famname* specify the clone.

11.2 Tools

Many utility programs are included with Habitat. The following sections provide a brief overview of their functionality and detail some commonly used functions.

11.2.1 hdbcloner

This utility is used to create, update, and remove clones. The syntax is described below:

hdbcloner -c *command* [-a *appname* -f *famname*]

Some commands include:

- show_clone: Shows a clone's status. The optional qualifier "-full" can be used.

- create_clone: Creates a new clone. Optional qualifiers include:
  - "-replicate" to mark the clone for replication
  - "-l" to specify a location other than HABITAT_CLONES
  - "-replace" to replace an existing clone.
- remove_clone: Removes an existing clone.

For more help using HdbCloner, refer to the *Hdb User Guide*. Help can be obtained from the command line via:

hdbcloner -h

For help with a specific command, use the syntax:

hdbcloner -c *command* -h

11.2.2 HdbCopydata

This utility is used to create and retrieve savecases.

hdbcopydata [–sf sourcefile] [-df destinationfile] [-s app.fam] [-d app.fam]

- Use the "-sf" qualifier to specify the full name (including the path) of the archive or savecase to be retrieved.
- Use the "-df" qualifier to specify the full name (including the path) of the archive to be created.
- Use the "-d" qualifier to specify the destination clone for the savecase.
- Use the "-s" qualifier to specify the source clone from which to create the savecase.

Note that data can be copied between clones using "-s" to specify the source clone and "-d" to specify the destination clone.

For more help with HdbCopydata, refer to the *Hdb User Guide*. For command-line help, use the syntax:

hdbcopydata -h

11.2.3 HdbRio

This utility is used to view or modify the contents of a database. To invoke HdbRio, the default context must be set to a valid clone, or the command-line qualifier to specify the clone must be used.

hdbrio [-a application] [-f family] database

Once at the hdbrio prompt:

- Use the "list" command to show the database's record structure.
- Use the "pos *record=subscript*" command to position to a specific record.
- Use the "find *record=key*" command to position to a record by searching for a particular value of a record's ID field.
- Use the "read" command to display the contents of a record.
- Use the "/*field_record(subscript)*" command to view a particular field value. The syntax has three valid values. The subscript can specify:

- A single record identified by an integer, e.g., /id_substn(23)
- A range of records defined by start_subscript:end_subscript, e.g., /id_substn(2:10)
- All records identified by the asterisk (*) wildcard, e.g., /id_substn(*)

Valid values for *subscript* are an integer, a range using *value:value*, or "*" for all subscripts. The value can be changed by adding "=*newvalue*" to the command. Use quotation marks to include special characters in a string.

- Use the "verify -a" command to validate the pointers in the database.
- Use the "reset" command to reset the pointers in the database.

11.2.4 HdbDump

This utility is a developer's debugging tool for analyzing Hdb binary file problems. It can also be used by a system administrator to determine whether various Hdb files are valid or corrupt.

The HdbDump utility is designed to dump the schema (when this applies), the structure, and the data of any type of Hdb binary file. This utility does not bother with ASCII coded files because they can be easily viewed using an ordinary text-viewing command.

Online help is available using the -h parameter switch.

```
hdbdump -h
```

11.2.5 Habitat Test Programs

Numerous test programs are delivered with Habitat. For a description of available test programs, refer to the appropriate user guide.

11.3 Common Problems and Questions

This section provides solutions to commonly encountered problems and answers questions about how to find specific information.

11.3.1 Habitat Fails to Start

There are several possible reasons for this to occur, depending on the platform.

For Linux, starting up Habitat does little more than starting hdbserver. Check that hdbserver started successfully by looking in TVIEW. If hdbserver failed to start, examine its log file in habitat_logdir/hdbserver.log.

Most problems can be uncovered by running hdbserver interactively with the "-verbose" option.

11.3.2 How to Prevent Habitat from Rebooting the Local CPU

Both PROCMAN and Cfgman use the same facility to reboot the local CPU in certain circumstances. This facility can be prevented from actually rebooting the machine by defining the following environment variable:

```
OSAL_REBOOT_DISABLED = true
```

> **Note:** Cfgctrl and PROCMAN must be started with this environment variable defined. If the environment variable is defined after the tasks have started, it has no effect.

11.3.3 DEBUG Environment Variables

For a list of available debug environment variables, refer to the *Habitat Software Installation and Maintenance Guide*.

11.3.4 How to Move Clones

Clone files can be moved only when Habitat is shut down. Once Habitat has been stopped, copy the clone file to the desired directory and delete the original. Before restarting HdbServer, regenerate Hdb's clone directory with the command:

```
habitat clone -p reset -dir clone_directory,clone_directory
```

where *clone_directory* is the complete path name of each directory on the system that contains clone files. Failure to include a directory means that the clone is unavailable to the online system.

11.3.5 Where can I find the version of Habitat?

The current Habitat version can be displayed by examining the habitat_version.dat file located in the HABITAT_CFGDIR directory.

11.3.6 Can I modify the database size of a Habitat-owned application?

Most Habitat applications are not designed to be dimension-independent with respect to database sizing. Therefore, it is not possible to resize a Habitat–owned application.

Hdb provides API functions to allow you to write dimension-independent applications.

11.3.7 Why is my recovery set not getting recovered?

One fairly common situation when setting up a dual-redundant Habitat is that one or more recovery sets are not recovered when the Standby machine becomes enabled, despite the fact that they are defined in CFGCTRL and CFGMONI databases and they do appear to be in service on the RECOVERY,CFGMONI display.

To solve such problems, the steps listed below should be followed:

1. Make sure that the replication becomes available and that some recovery sets are recovered. If this is not the case, verify that all Cfgman and MRS tasks are running on both computers. For details, check the *Configuration Manager User Guide*.
2. Check the Habitat logs used by Cfgman and MRS (available from CFGCTRL displays) on both computers. The answer may be somewhere in the logs.
3. Verify that the recovery set having a problem is defined in the CFGCTRL and the CFGMONI databases on both computers.

4. Verify that the associated clone has been created as “replicated”. This check should be done on both computers with the following command:

   hdbcloner -c show_clone -a *appname* -f *famname* -verbose 2

   At least one database should appear flagged as “REPLICATED”.

5. Verify that both clones on each computer have the same definition stamp for the database(s) being replicated. To view the definition stamps, use the command:

   hdbcloner -c show_clone -a *appname* -f *famname* -verbose 4

6. If all of the above did not solve the problem, stop Habitat on the Standby machine and copy the clone to be replicated from the Primary machine to the Standby. Then restart Habitat.

12. Linux-Specific Issues

This chapter contains information specific to the Linux implementation of Habitat. Any information in this chapter supersedes information elsewhere in the document.

12.1 Changing the Maximum Number of Clones

By default, Hdb permits only 150 clones to be active at one time. In a Linux system, changing the maximum number of clones supported by Hdb involves the following steps:

1. Shut down Habitat with the "habitat stop" command.

   > **Note:** It is vital that this command and only this command be used for this activity. There are temporary files that are deleted by this procedure that otherwise prevent the resizing of the maximum number of clones supported.

2. Modify the habitat_start_tasks.sh procedure. Remove the "tview" command to run HdbServer and replace it with:

   context hdb habitat

   hdbserver -mxclones *max_clones* &

   After restarting Habitat, the maximum number of clones corresponds to the value specified for *max_clones*.

Another way to set the maximum number of clones is to define the environment variable HABITAT_MXCLONES with the maximum clone number.

In Linux, a potential issue when increasing the number of clones has to do with OSAL shared-memory implementation. For more information about issues with increasing the number of clones in Linux, refer to the *Habitat Software Installation and Maintenance Guide*.

12.2 Understanding the Habitat UPDATE Process

Changing Habitat environment variables on Habitat systems running under the Linux operating system requires Habitat to perform several steps. This section describes these steps, to provide a background for working with environment variables in Linux.

Habitat attempts to validate all critical environment variables needed for correct operation. However, since this validation procedure is time-consuming and may detect errors, Habitat does not perform it each time a user logs on. Instead, the validation is performed once during the "habitat update" process and the resulting valid set of environment variables is written to a file. This file is located in HABITAT_MANAGER and called "habitat_environment_*host_group*.csh", where *host* is the name of the system and *group* is the Habitat group number in hex. Since this file is used to define the Habitat environment variables, it is possible to modify the environment by editing the file directly. However, this is not recommended. Habitat can operate unpredictably or incorrectly if these environment variables are not correct.

Instead, to modify the Habitat environment, modify the habmgr_define_environment.csh file located in HABITAT_MANAGER. As previously described, this file is divided into three parts. Use the first part to define custom values for Habitat environment variables. The second part is owned by Habitat, and the third part is used to define environment variables that are based on (but not owned by) Habitat.

When the “habitat update” command is issued, the sequence of events is as follows:

1. Habitat makes a backup of the existing habitat_environment_host_group.csh file, renaming it with a .bak extension.

   The original is then deleted.

2. Habitat invokes a procedure to regenerate the environment file using the “rsh” command.

   This creates a subprocess that goes through the normal login procedure but does not inherit any existing environment variable definitions. This prevents Habitat from picking up local customizations of the environment.

   Only values defined in habmgr_define_environment.csh are seen by the procedure that re-creates the environment file.

3. The procedure invoked performs validation work on the Habitat environment. This includes checking the validity of all customizations of Habitat environment variables defined in the first section of the habmgr_define_environment.csh file.

   Any value not specified in that file is defaulted to its correct location (based on information placed in the habmgr_define_environment.csh file at installation time). If this validation is successful, a new habitat_environment_host_group.csh file is created.

4. The procedure displays a message, indicating that the new environment will not be picked up until the next time you log in.

5. If the validation fails, correct the problem in habmgr_define_environment.csh and perform the update again.

   If necessary, the backup of the environment file can be renamed to restore the starting environment.

13. Windows-Specific Issues

This chapter contains information specific to the Windows implementation of Habitat. Any information in this chapter supersedes information elsewhere in the document.

13.1 The Habitat Product

Habitat in Windows is slightly different than in Linux because of two key factors:

- Habitat's user interface is provided in Windows only. Thus, there are components of Habitat that are only delivered and installed in Windows.
- Habitat in Windows serves as the primary development platform for developers at GE. Development time is considerably reduced when using Habitat in conjunction with Microsoft Visual Studio.NET. This same versatile and efficient environment is now available to customers who are developing Habitat applications.

The following points highlight the differences between Habitat in Windows and in other operating systems. The topics highlighted here are expounded upon in later sections or in the *Habitat Software Installation and Maintenance Guide*.

- Windows-based Habitat can be installed as either a shared service or an application. When installed as a service, Habitat does not require that any user be logged on in order to be running, and it also allows interaction by multiple simultaneous users. When installed as an application, Habitat is completely conditional upon the existence of a single logged-on user account.
- Habitat in Windows is installed via an InstallShield installation kit.
- When Habitat is installed in Windows, it is ready to run and does not require any additional building or modifications.
- In Windows, logging in requires that each user log in with a unique user account.
- Customizing environment variables is different in Windows. For details, see section 13.3.2 Environment Variables in Windows.
- Different tools are used to build and customize Habitat applications in Windows. Microsoft Visual C++ and Intel Visual Fortran are required if you are building against Habitat libraries.

13.2 Habitat as a Service

This section discusses running Habitat in the Windows service environment.

13.2.1 Overview

Windows has different "environments" in which an executable file can run. One environment is associated with a user's login. In this environment, a user starts applications, generally by double-clicking icons, and stops applications, either explicitly or by logging off. The applications run in the user's environment and have complete visibility in that environment — but in no other user's environment. When the user's logoff terminates the environment, all applications started by that user also terminate.

In addition to the user environment, Windows also has a service environment. This particular environment is initialized when the system boots. Windows can be configured to automatically start services at boot time. These services remain active completely independent of users logging on and logging off.

Services are controlled via Windows' "Service Control Manager" (SCM). This manager is responsible for starting and stopping all services. The Service Control Manager processes requests received programmatically, or from direct interaction via the Services dialog box[2]. A user with the appropriate authority can start and stop services via the Services dialog box. If a user does not explicitly shut down a service, it remains running until the system is rebooted (or until it shuts itself down).

Services depend on the Service Control Manager. Of importance to Habitat is the fact that services receive all of their environment information from the Service Control Manager, which reads the user-level environment when the service is started. Any changes to the user-level environment require the stopping and restarting of the service.

Habitat can be installed to run in the Windows service environment. This allows a single instance of Habitat to service multiple users simultaneously and to persist throughout user logons and logoffs. This form of Habitat is typically used for a production environment and not for a development environment.

In order to exist in this environment, a service application must be installed and started via the Service Control Manager. The installation of the service occurs after the Habitat installation (see the *Habitat Software Installation and Maintenance Guide*). Starting and stopping the service can be controlled via the Start menu shortcuts, using the Windows Control Panel > Services dialog box, or via the "net start" and "net stop" commands provided by Windows[3]. Habitat can also be configured to start during system boot via the Services dialog box.

After the Habitat service is started, TVIEW can be started. Requests from TVIEW are processed with regards to the environment that the Service Control Manager established at the starting of the service. Thus, a command such as:

```
> tview run procman procman habitat
```

causes the Habitat service to start PROCMAN in the service environment. All tasks that PROCMAN then starts are started in the same service environment.

For more details about TVIEW, see section 13.2.5 TVIEW.

13.2.2 Advantages

- Installing Habitat as a service provides persistence. A Habitat service can be started either at boot time or via user-issued commands. Once the service is started, the Habitat applications can be started. This process can be automated using the technique described in section 13.5 Configuring Habitat to Start after Rebooting. After the service has been started, all user accounts can be logged off and Habitat remains running.
- Multiple users can interact with Habitat without the need to customize each user account. A user can log in, interact with the Habitat service, and log off — permitting a different user to log in and interact with the same instance of Habitat. Or, while one user is logged in, another can log in via TELNET, and both users can simultaneously interact with Habitat. Additionally, if installed in Windows Terminal Server Edition, multiple users can access the same instance of Habitat via a

[2] Accessed via the Control Panel.
[3] Some information about the NET command can be accessed by typing "net /?" at the command prompt.

standard Windows GUI interface (for additional information, see section 13.3.2.2 Multiple Instances of Habitat.

- Because the Service Control Manager establishes its environment from the user environment, multiple Habitat services can be installed on a given system. This Habitat instance can co-exist with other non-service Habitat instances and is constrained by the same rules as discussed in section 13.3.2.2 Multiple Instances of Habitat.

13.2.3 Habitat Accounts and Groups

Habitat groups in Windows are treated the same as they are in Linux. However, accounts behave a little differently when Habitat is run in the Windows service environment.

For Habitat to be run as a service, a user account must be created under which it runs. This account must be created prior to installing Habitat, and it must be granted the "log on as a service" privilege prior to starting Habitat.

All accounts have access to the service if they are associated with its group number. There are some special considerations for this under the Terminal Server environment. For more details, see section 2.3 Habitat Environment Variables in Chapter 2.

A final important concept is that of a user environment and its association with the account. Each user account establishes its environment based on the Windows registry values at login time. For more details, see section 2.3 Habitat Environment Variables.

13.2.4 Starting and Stopping

The only difference regarding starting and stopping a service mode of Habitat is that it must be the first thing started. This can be accomplished in these ways:

- You can configure the Service Control Manager to start the service mode automatically at boot time.
- A user can start or stop it directly using the Services dialog box.[4]
- A user can issue a "net" command[5] (note that shortcuts are created for the "net" commands during the Habitat installation).

After the service is started, TVIEW is used to perform the same tasks it was used for when running in the non-service mode.

When the service is stopped, all Habitat tasks that have been registered with the service are stopped. This includes any Habitat tasks that were started while the service was running.

[4] Accessed via the Control Panel.
[5] Information about the NET START and NET STOP commands can be found by typing "net /?" at the command prompt.

13.2.5 TVIEW

The text mode version of TVIEW in Windows can be used to interact with Habitat in both service and non-service modes. If the Habitat service is not running when a “start” command is issued, TVIEW issues the following warning:

> Tasks you start will not be persistent - See “About”

This warning is simply to advise a user that the Habitat service is not running or not installed. Tasks started in this manner are not persistent; they terminate when the current user logs off.

The TVIEW GUI does not issue warnings. However, it does have an indicator in the status bar that identifies if the TVIEW is acting in the SERVICE mode (when Habitat is running as a service) or the NON-SERVICE mode (when Habitat is not running as a service).

Regardless of the mode in which Habitat is running, both versions of TVIEW interact with it in the same way. If Habitat is running as a service, TVIEW sends all requests to the service, which executes them via the APPMGR API.

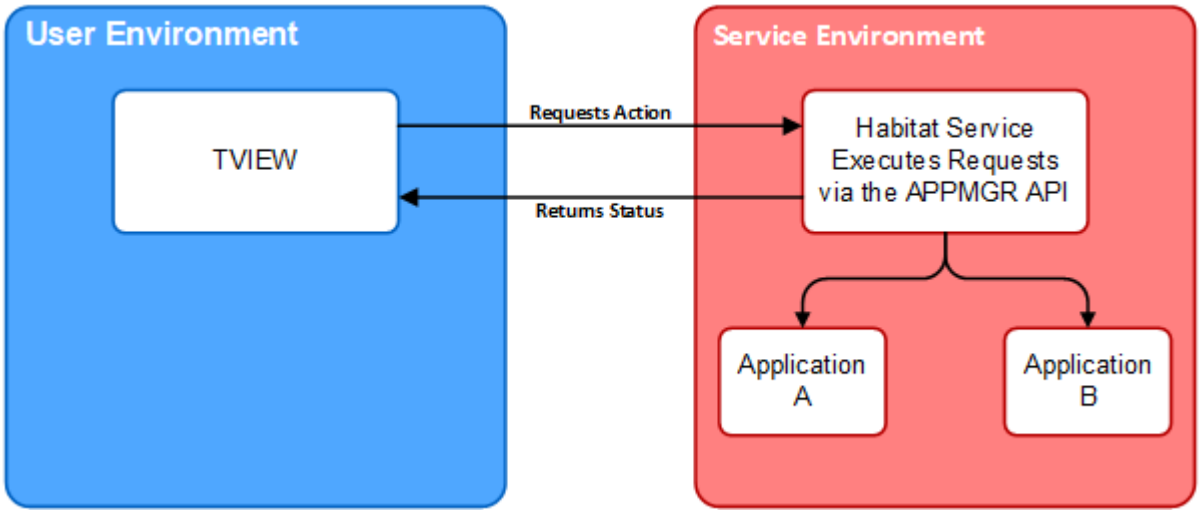

Figure 14. TVIEW Interacting with Habitat Running as a Service

When Habitat is not running as a service, TVIEW executes the commands directly via the APPMGR API.

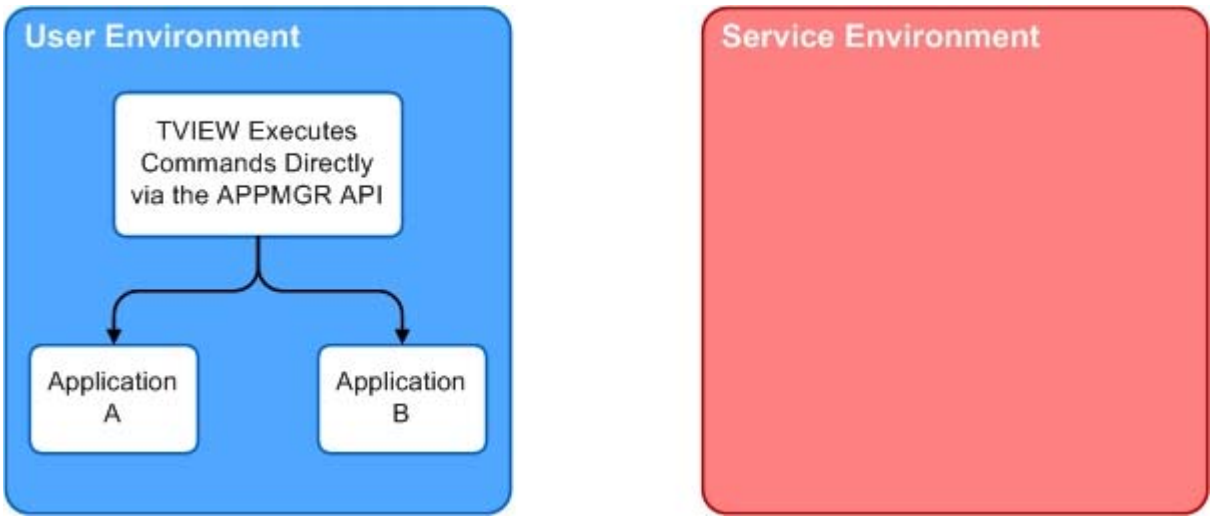

Figure 15. TVIEW Interacting with Habitat Not Running as a Service

13.3 Habitat Environment Variables

This section describes Habitat environment variables in Windows and where they differ in use and management from Linux.

13.3.1 Overview

Habitat environment variables in Windows are used in much the same manner as they are on any other platform. However, their management and customization differ slightly in Windows. The following sections discuss those differences and how environment variables are managed in Windows.

13.3.2 Environment Variables in Windows

In Windows, environment variables are stored in the Windows registry. Windows categorizes environment variables into SYSTEM-level and USER-level areas. Variables set at the system level are visible to all users who log on to a given PC, while variables set at the user level are only visible to the particular user account for which they are set. When a user logs on, variables set at the user level override variables set at the system level.

The PATH environment variable is handled differently than all others. The PATH variable at the user level does not override the PATH variable at the system level. Instead, the user-level PATH variable is concatenated to the system-level PATH variable.

When Habitat is installed, it sets all of the environment variables that it requires during the installation process under the user level. The installation procedure creates a batch file that can be used to restore the environment registry settings if they are lost; this file can also be used as a template for customization. The file is named habitat_define_user_env.bat and is generated in the HABMGRGG\habmgr directory.

13.3.2.1 Customizing Environment Variables

Normally, there is no need to customize environment variables that the installation program sets. However, if you change your Habitat configuration, move Habitat to another disk location, or require any other customizations, the environment variables can be modified by accessing the Windows Control Panel > System > Environment. From this point, you can add, remove, and modify environment variables as necessary.

When modifying Habitat variables, you normally set them at the user level, unless Habitat was installed as a service.

> **Note:** Changes via the environment panel persist after a user has logged off. However, *applications or command prompts running or open when the changes are made do not apply these new values until they are restarted.*

For temporary changes to environment variables, you can issue the "set" command from a command prompt.

Note that these changes are not applied to any application currently running, or to any application that is not subsequently started from the command prompt where the change was made. Using the "set" command affects only the session from which it was issued and the child processes it may create. Additionally, these changes are not persistent and are lost when the command prompt is closed. For example:

```
> set timedate_config_file=%HABITAT_CFGDIR%\test.tmdt
```

Following this command, any tasks started from the current command prompt use this new value (including tasks started by TVIEW).

Finally, some site administrators might not want to have a large number of environment variables set in the Windows registry. It is possible to create a batch file that establishes the Habitat environment as part of the startup procedure. The batch file generated during installation, which is located in the HABMGRGG\habmgr directory, can be used as the basis for this technique. However, this technique is not currently supported for a Habitat service installation.

13.3.2.2 Multiple Instances of Habitat

A workstation/server can have more than one instance of non-service Habitat installed at a given time. However, due to constraints imposed on Windows services, there can only be a single service-type installation.

For multiple instances of Habitat, the environment variables are set at the user level. This environment table is read each time a particular user logs on. This prevents access to this instance of Habitat by other user accounts. It also enforces the restriction that Habitat must be installed from the account that intends to use it. Thus, an administrator account cannot be used to install Habitat for another account.

13.4 TVIEW

Habitat in Windows includes a version of TVIEW that is equivalent to the version shipped with Linux. All "tview" commands discussed throughout this document apply to Windows as well.

The common version of TVIEW can be launched from the command prompt by typing:

> TVIEW

It can be also launched from the Start menu folder:

TVIEW (Standard)

The Windows version of TVIEW has one significant difference from its Linux counterpart: it can interact with Habitat running as a service. As such, the main menu has an additional menu item, "(A)bout Service Mode". This option briefly describes the service mode that TVIEW can operate in. Additionally, when not running in service mode, TVIEW issues a warning that the started task is not persistent the first time an application is started with the current TVIEW session. For a non-service type Habitat, all tasks are terminated when the user logs off. For more details, see section 13.2 Habitat as a Service.

In addition to the TVIEW with a command-line interface, Habitat in Windows provides a TVIEW GUI application. This application performs the same commands as the command-line version but provides a Windows user interface.

Use the following command to start the TVIEW GUI:

> TASKVIEW

It can also be launched from the Start menu folder:

TVIEW (GUI)

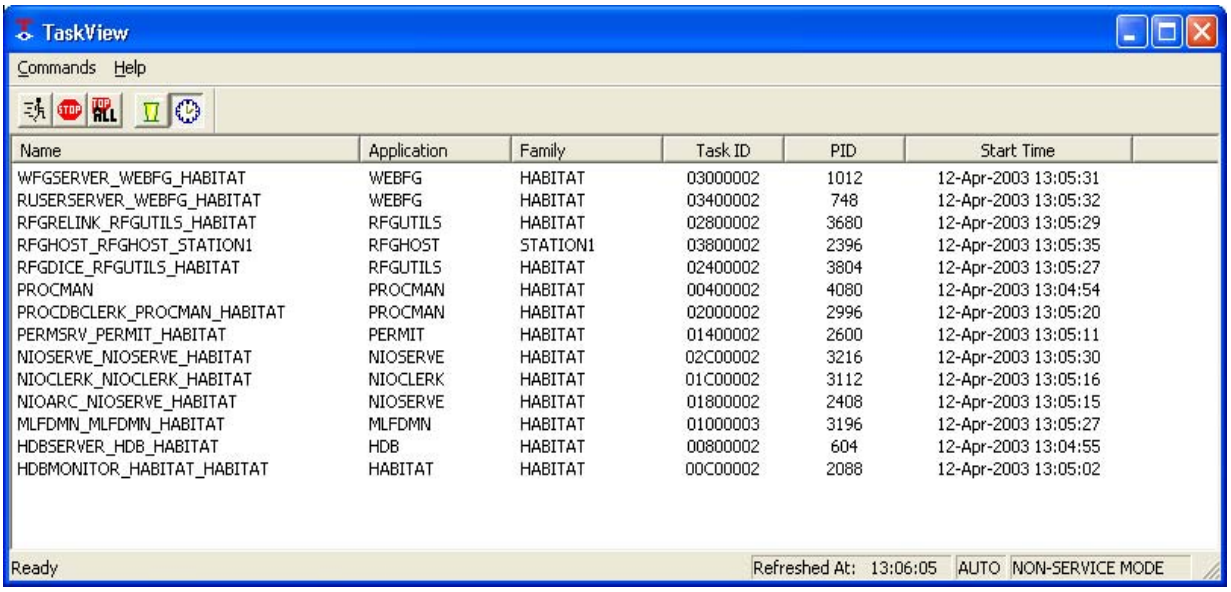

Figure 16. TVIEW Graphical User Interface

Status bar fields:

- **Status:** Displays "Ready" or the last status reported by APPMGR after you issue a command.
- **Refreshed At:** Displays the last time the display was refreshed. This time is based on the local system clock and does not use the Timedate API.
- **Auto:** This field indicates when Auto Refresh is enabled. When Auto Refresh is disabled, the field is blank.

- **Service Mode:** This field indicates whether TVIEW is running in SERVICE MODE or NON-SERVICE MODE.

The main menu items are shown in Figure 17:

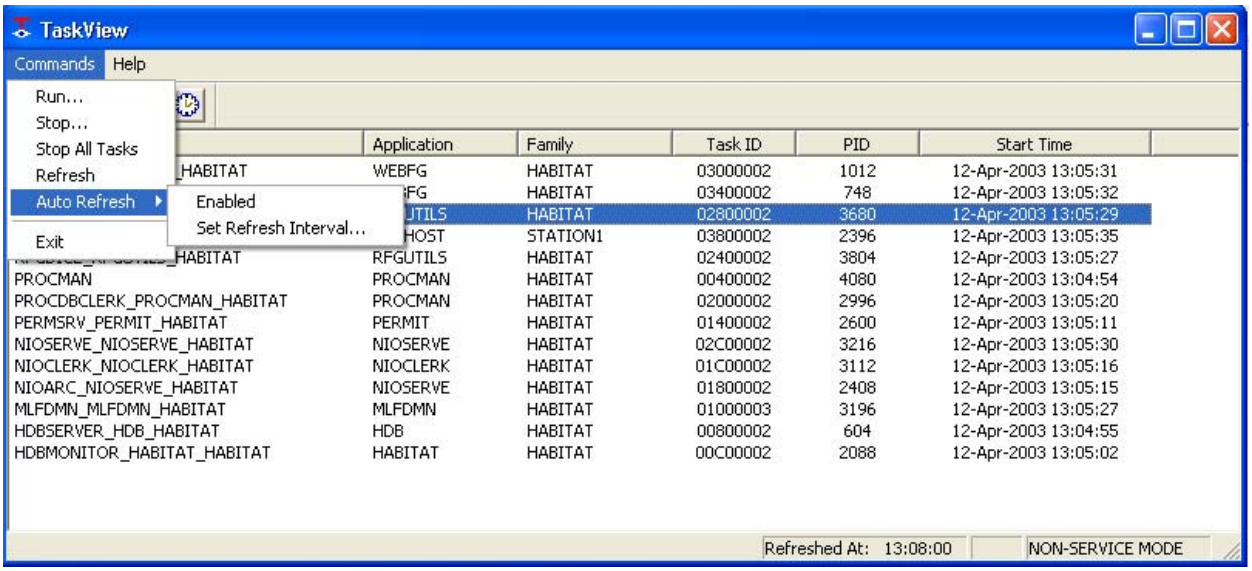

Figure 17. TVIEW Main Menu

Description of menu items:

- Commands: The Commands menu item allows you to:
  - Run a task
  - Stop the highlighted task(s)
  - Stop all tasks
  - Refresh the display
  - Enable/disable Auto Refresh
  - Modify the Auto Refresh rate
  - Exit TaskView
- Help: Displays copyright and version information

Description of tool bar:

- Running Person: Starts or “runs” an application
- Stop: Stops the selected application(s)
- Stop All: Stops all applications
- Lemonade: Refreshes the display
- Timer: Enables/disables Auto Refresh

Additionally, right-clicking displays a menu with the same functionality as the Command menu.

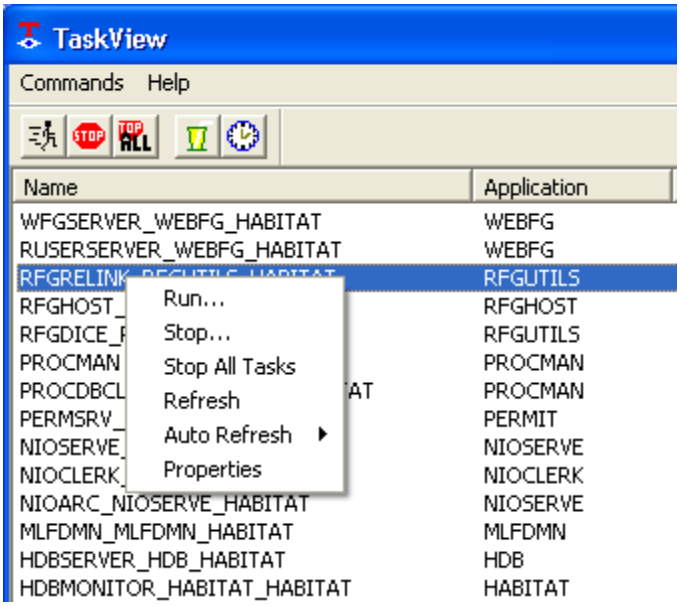

Figure 18. TVIEW Right-Click Menu

13.4.1 Using Shortcuts to Automate Startup

To start the user interface applications automatically when a user logs on, shortcuts can be created in the startup folder of the Start menu. Use the Browse option to create the shortcut in the Start Menu > Programs > Startup folder.

13.4.2 Special Privilege to run TVIEW on Windows

In Windows, a user running Habitat needs a special privilege named CreateGlobalObjects. If the account does not have this privilege, you see the following error message when you try to start TVIEW:

```
OSAL MEMSHARE - map_write - CreateFileMapping Failed
    shared area name = 'appmgr_registry'
    shared area desc = APPMGR Task Registry shared memory area
    CreateFileMapping error = 00000005
```

For more information about setting up this privilege, see the *Habitat Software Installation and Maintenance Guide*.

13.5 Configuring Habitat to Start after Rebooting

It is possible to configure the system to start Habitat after a reboot. To automatically start Habitat without configuring it as a service, follow these steps:

1. Modify the Windows registry: Navigate to HKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\Current Version\Winlogon
2. Add a new value: AutoAdminLogon REG_SZ

   Range: 0 or 1

AutoAdminLogon specifies automatic logon if this value is 1.

3. Add a new value: DefaultUserName REG_SZ

   DefaultUserName specifies the name of the last successfully logged-on user. If values are defined for DefaultPassword and AutoAdminLogon, this is the user who is logged on by default during automatic logon. This value is dynamic and changes with each logon. Thus, for AutoAdminLogon to work, you need to ensure that the desired account was the last account logged on.

4. Add a new value: DefaultPassword REG_SZ

   DefaultPassword specifies the password for the user listed under DefaultUserName.

   When AutoAdminLogon is used, Windows automatically logs on the specified user when the system is started, bypassing the Logon Information dialog box.

5. Construct a batch file: This file is used to start the Habitat tasks and may be as simple as creating a batch file that includes the line:

   tview run procman procman habitat

6. Place a shortcut to the batch file in the user's startup folder.

The steps described above involve modifying the registry and saving a password, so it is not the ideal approach.

A better approach to restarting Habitat after a reboot is to configure it as a service, and make the service start automatically after a reboot. The details of configuring the service are described in the *Habitat Software Installation and Maintenance Guide*.

13.6 Non-Standard Network Considerations (Laptops)

This information is provided for non-standard installations. Issues have arisen when running Habitat on laptops that are configured for networks but are not always connected to a network.

A laptop — or any workstation that runs Habitat while disconnected from the network — should have an entry made in its host file indicating the IP address of the PC. When disconnected from a network, the TCP/IP services are unable to access the TCP/IP Distributed Name Service to look up addresses of hosts. Because of this, NETIO communications are not established.

To resolve this, define the PC name with its correct IP address in the local host file. This file should be located in the \WINNT\system32\drivers\etc folder. The file should be \WINNT\system32\drivers\etc\hosts.

The following is an example of the line to insert in the host file:

199.100.10.10 PcName

If the laptop/PC supports multiple boot configurations such as both a dial-up adapter and a standard network adapter, and if the PC is <u>not</u> connected to the network, the network adapter configuration should be used.

By way of explanation, if the PC uses some tunneling protocol for dial-up connections, it is not assigned an IP address until its connection to the network is established (DHCP). So, if the dial-up boot configuration is used, and if the PC is never connected to the network, even modifying the host file is not sufficient to allow NETIO connections to be formed. Using the standard network adapter and modifying the host file allows NETIO to form connections.

A simple check to ensure that your PC is configured properly is to open a DOS window and ping the PC name when the computer is disconnected from the network. If the PC replies and displays the correct IP address, Habitat runs. If the ping responds that the host is unreachable, the host file must be corrected.

A more-thorough check can be performed with the NETIO Configuration Check Program (NIOCONFIG). The NIOCONFIG program checks to see how the local machine is configured from a network and a NETIO perspective. It is self-documented and can be run from the command line with the following command:

```
Nioconfig
```

Index

A

accounts, 8
Alarm, 57, 59, 65
alarm message files (.almmsg), 43
application
  defined, 4
  setting the context for commands, 22
application definition files (.appdef), 46
archives, 29, 30

B

Browser, 3, 5
BUILD command, 39

C

CLONE command, 42
Cloner. See HdbCloner
clones
  creating, 27, 66
  defined, 4
  maximum number, 45, 71
  removing, 28
  schema files, 4
  updating, 28
common problems and questions, 68
context
  defined, 4
  setting, 22, 66
creating clones, 27, 66
customizing
  Habitat environment variables, 17
  login procedure, 19
  savecases, 63
  startup, 17

D

database subsystem (Hdb), 5
Display Builder, 5
Display Definition Language (DDL), 5

E

environment variables, 11, 77

F

family
  defined, 4
  setting the context for commands, 22
file protection and ownership, 12

G

getenv command, 22
groups, 7, 8

H

Habitat
  architecture, 7
  directory structure, 8
  environment variables, 11
  environment variables, customizing, 17
  HABITAT PROCMAN model, 49
  login procedure, customizing, 19
  overview, 2
  portability, 1
  PROCMAN subsystem, 49
  programming environment, 6
  running as a service, 73
  running on laptop computers, 82
  run-time environment, 5
  starting, 21, 66
  stopping, 21, 66
  troubleshooting, 68
  user interface development environment, 5
habitat commands
  BUILD, 39
  CLONE, 42
  HELP, 39
  LINK, 40
  LOAD, 41
  overview, 38
  START, 38
  STOP, 38
  UPDATE, 39
HABITAT PROCMAN model, 49
habitat_define_user_env.bat, 77
Hdb (database subsystem), 5
Hdb server (HDBSERVER), 49, 68
HDB server monitor (HDBMONITOR, 49
HdbCloner utility, 4, 26, 30
HdbCopydata utility, 4, 28, 29, 30
HdbDump utility, 68
HdbFormat program, 4
HdbRio utility, 32, 67
HELP command, 39

L

laptop computers, running Habitat on, 82
LINK command, 40
listmsg utility, 25
LOAD command, 41

localizing messages, 47
LOGMAN, 60

M

maximum number of clones, 45, 71
Memory Replication Service (MRS), 54
message logs, viewing, 25
messages, localizing, 47

N

NETIO, 59
NIOCONFIG configuration check program, 83

O

OSAL_REBOOT_DISABLED environment variable, 68
ownership, files, 12

P

PLC Manager, 61
portability, 1
PROCMAN clerk (PROCDBCLERK), 49
PROCMAN subsystem, 49
programming environment, 6
protection, files, 12

Q

quotas and privileges, 46

R

recovery, 53, 54
removing clones, 28
replication, 53, 54
Reset function, 36
retrieving cases, 66
run-time environment, 5

S

Sampler, 60
savecases, 4, 29, 30, 31, 63
shutdown, 14
Software Configuration Management (SCM), 62
START command, 38
starting Habitat, 21, 66
startup, 14
status files (.status), 43
STOP command, 38
stopping Habitat, 21, 66
supported software configurations
  defined, 69
system build, 43

T

troubleshooting, 68
tview (task viewer) utility, 23

U

UPDATE command, 39
updating clones, 28
user interface development environment, 5
utilities
  HdbCloner, 4, 26, 30
  HdbCopydata, 4, 30
  HdbDump, 68
  HdbRio, 32, 67
  listmsg, 25
  tview (task viewer), 23

V

Verify function, 36